



Sanjivani College of Engineering, Kopargaon

Department of Computer Engineering

(Affiliated to Savitribai Phule Pune University, Pune)

A Project Report on

NextGen Rakshak

Smart Edge-Based Lost Child Recovery System for Mass Gatherings

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

in

Computer Engineering

Submitted by

Bankar Smitraj Dinkar	Roll No. 09
Bhakare Tanishka Sharad	Roll No. 11
Dhadge Vedant Sanjay	Roll No. 34
Narkhede Atharva Anantkumar	Roll No. 94

Group ID: 6

Under the guidance of

Dr. A. B. Pawar

**Department of Computer Engineering
Sanjivani College of Engineering, Kopargaon**

Academic Year 2026–2027

Sanjivani College of Engineering, Kopargaon

Department of Computer Engineering

CERTIFICATE

This is to certify that the project report entitled

“NextGen Rakshak: Smart Edge-Based Lost Child Recovery System for Mass Gatherings”

has been submitted by

Bankar Smitraj Dinkar	Roll No. 09
Bhakare Tanishka Sharad	Roll No. 11
Dhadge Vedant Sanjay	Roll No. 34
Narkhede Atharva Anantkumar	Roll No. 94

in partial fulfilment of the requirements for the degree of **Bachelor of Technology in Computer Engineering**, for the academic year 2026–2027, as a record of bona fide work carried out by them under my/our supervision.

Dr. A. B. Pawar
Project Guide

Dr. S. R. Deshmukh
Project Coordinator

Dr. M. A. Jawale
Head, Department of Computer Engineering

External Examiner

Place: Kopargaon

Date: September 13, 2026

DECLARATION

We hereby declare that the project entitled “**NextGen Rakshak: Smart Edge-Based Lost Child Recovery System for Mass Gatherings**”, submitted for the degree of Bachelor of Technology in Computer Engineering, is our own original work. It has been carried out under the guidance of **Dr. A. B. Pawar** and has not been submitted, in whole or in part, for the award of any other degree, diploma, or similar title of this or any other institute. All sources of information used and all assistance received have been duly acknowledged. Where AI-assisted tools were used during documentation drafting, literature synthesis, and report formatting, this is explicitly disclosed here; all system design, architecture decisions, and implementation are the team’s own work.

Bankar Smitraj Dinkar (09)

Bhakare Tanishka Sharad (11)

Dhadge Vedant Sanjay (34)

Narkhede Atharva Anantkumar (94)

Place: Kopergaon

Date: September 13, 2026

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to our project guide, **Dr. A. B. Pawar**, for his continuous guidance, valuable suggestions, and encouragement at every stage of this project, from the initial problem framing through the measurement of the face-matching threshold to the final report. His insistence on measuring claims rather than assuming them shaped the engineering discipline behind this work.

We are grateful to our Project Coordinator, **Dr. S. R. Deshmukh**, for the structured review checkpoints that kept a four-person, three-codebase project on schedule, and to our Head of Department, **Dr. M. A. Jawale**, for providing the departmental infrastructure and encouragement necessary to pursue a project of this scope.

We also thank the faculty of the Department of Computer Engineering, Sanjivani College of Engineering, Kopargaon, for their support throughout the final year, and our families and peers who volunteered their photographs and their patience during repeated field trials of the offline mesh network.

Finally, we acknowledge the open-source and public research communities behind Google ML Kit, TensorFlow Lite, MobileFaceNet, Firebase, and the Nearby Connections API, whose published work made a project of this ambition achievable within a single academic year by a four-member student team.

Bankar Smitraj Dinkar, Bhakare Tanishka Sharad,
Dhadge Vedant Sanjay, Narkhede Atharva Anantkumar

ABSTRACT

India reports tens of thousands of missing children every year, and both law enforcement and child-rights researchers agree that the first 60–90 minutes after a child is separated from their guardian – the “golden hour” – are the most critical for recovery. This window is precisely when mass gatherings such as Kumbh Mela, Dussehra processions, and Ganesh Chaturthi visarjans are at their most congested, and cellular networks at such events are frequently saturated, making the internet-dependent, centralised, and reactive “upload-and-wait” architecture of existing systems (TrackChild [2], Khoya-Paya [3], ReUnite [4]) structurally unsuited to the problem.

NextGen Rakshak is a hybrid edge-AI system designed around a different premise: rather than uploading a bystander’s face to a central server for matching, every recognition decision is computed *on the volunteer’s own smartphone*. A police officer files an alert with a photo and details at a web-based kiosk portal; a Cloud Function computes a face embedding and geofenced push notification carries it to nearby, pre-registered volunteers (police, NCC/NSS cadets, NGO staff, shopkeepers) within seconds. Each volunteer’s Android app re-computes the embedding on-device, detects and aligns faces live from the camera using Google ML Kit, and compares them against the alert using a MobileFaceNet (TensorFlow Lite) embedding and cosine similarity – with a human volunteer always required to visually confirm a candidate match before it is reported. No bystander image, video frame, or biometric embedding ever leaves the phone.

When cellular connectivity is degraded or absent altogether, a custom multi-hop store-and-forward mesh, built on top of Google’s Nearby Connections API (Bluetooth Low Energy discovery, Wi-Fi Direct transfer), propagates alerts and confirmed sightings phone-to-phone with HMAC-authenticated packets, time-to-live hop limiting, and duplicate suppression – keeping the system operational exactly when the network is least available.

The system was implemented as three integrated components – a Next.js police kiosk portal, a Kotlin/Jetpack Compose volunteer Android application, and Firebase Cloud Functions for coordination – and validated on real Android hardware, where the measured face-recognition threshold (cosine similarity > 0.55 , empirically derived rather than taken from literature) produced a genuine, on-device, end-to-end match. This report documents the requirement analysis, system design, implementation, and evaluation of the system, and honestly states which non-functional targets remain to be measured at scale.

Keywords: lost-child recovery, edge AI, on-device face recognition, MobileFaceNet, offline mesh networking, Nearby Connections, privacy-by-design, golden hour, mass-gathering safety.

Contents

1	Introduction	1
1.1	Problem Definition	1
1.2	Literature Review / Relevant Theory	2
1.3	Scope	4
1.4	Objectives	5
2	Requirement Analysis	6
2.1	Requirement Specifications	6
2.1.1	Normal Requirements	6
2.1.2	Expected Requirements	6
2.1.3	Exciting Requirements	6
2.2	Validation of Requirements	7
2.3	Functional Requirements	7
2.4	Non-Functional Requirement	8
2.5	System Requirements	9
2.5.1	Hardware Requirements	9
2.5.2	Software Requirements	10
3	System Model	11
3.1	Process Model	11
3.1.1	Incremental Model	11
3.1.2	Why to use Incremental Model	11
3.1.3	Advantages	12
3.1.4	Disadvantages	12
3.2	System Breakdown Structure	12
3.3	Module Details	13
3.3.1	User	13
3.3.2	System Module	14
3.4	Project Estimation	14
3.4.1	Estimation in KLOC	14
3.4.2	Efforts	15
3.4.3	Development time in months	15
3.4.4	Total Time Required for Project Development	15
3.4.5	Number of Developers (N)	15
4	System Analysis	16
4.1	Project Scheduling and Tracking	16
4.1.1	Project Work and Breakdown Structure (Analysis)	16
4.1.2	Version Control and Commit History	16

4.1.3	Weekly Progress Log (Guide-Assessed)	18
4.2	Project Work Breakdown Structure (Implementation)	21
4.3	Task Identification	21
4.3.1	Ownership and Responsibility (as filed in the Project Proposal)	21
4.3.2	Project Schedule	22
4.4	Project Table and Time-line Chart	23
4.4.1	Project Schedule Time	23
4.4.2	Time-Line Chart	24
4.5	Analysis Modeling	24
4.5.1	Behavioral Modeling	24
4.5.2	Functional Modeling	27
4.5.3	Architectural Modeling	28
4.6	Mathematical Modeling	28
4.6.1	Venn Diagram	29
5	Risk Management	30
5.1	Risk Identification	30
5.2	Strategies Used to Manage Risk	32
5.3	Risk Projection	32
5.3.1	Preparing Risk Table	32
5.4	Feasibility	33
6	Technical Specification	35
6.1	Software requirement specification	35
6.1.1	Operating System (OS)	35
6.1.2	Integrated Development Environment (IDE)	35
6.1.3	Programming Language and Database	35
6.2	Hardware Requirement Specifications	36
7	Implementation Details	37
7.1	Firestore Data Model	37
7.2	Police Kiosk Portal	38
7.2.1	Alert Creation and the Kiosk Data API	40
7.2.2	Match Review and Dispatch	40
7.3	Volunteer Android Application	41
7.3.1	Face Recognition Pipeline	44
7.3.2	Alert embeddings are recomputed on-device	45
7.3.3	Face-Match Threshold: Measured, Not Assumed	46
7.3.4	Concurrency Safety on the Interpreter Thread	46
7.3.5	Local Persistence and Background Sync	47
7.4	Complete End-to-End Mobile Application Workflow	48
7.5	Cloud Functions and Data Coordination	55
7.5.1	Data Lifecycle – Privacy Purge on Resolve	57
7.6	Offline Mesh Networking	58
7.7	Security Implementation	60
7.8	Deployment	60
7.8.1	Police Kiosk Portal – Vercel	60
7.8.2	Cloud Functions, Firestore Rules, and Storage Rules – Firebase	61
7.8.3	Volunteer Android App – Signed Release APK	61

7.9	Testing, Field Validation, and Results	62
7.9.1	Testing Strategy	62
7.9.2	Unit Test Suite	62
7.9.3	System Test Cases	62
7.9.4	Field Trial – Offline Mesh on Three Physical Devices	64
8	Applications of the Project	66
9	Conclusion and Future Scope	67
9.1	Conclusion	67
9.2	Future Scope	67

List of Figures

3.1	System breakdown structure	12
4.1	Work breakdown structure – Analysis phase	16
4.2	Work breakdown structure – Implementation phase	21
4.3	Project time-line chart (weeks from Phase 1 start, 11 Aug 2026)	24
4.4	Use case diagram – Police Officer, Volunteer, and Cloud Functions	24
4.5	Activity diagram – alert filing through confirmed sighting	25
4.6	Sequence diagram – alert creation through match notification	26
4.7	DFD Level 0 – system context	27
4.8	DFD Level 1 – decomposition of core processes	27
4.9	Three-layer system architecture: Cloud, Edge, Mesh	28
4.10	Volunteer Android app – layered (SOLID) package architecture	28
4.11	Venn diagram – structural limitations of existing systems vs. NextGen Rakshak’s position	29
7.1	Police kiosk portal	39
7.2	Volunteer app – sign-in and alert browsing	42
7.3	Volunteer app – the core scan-and-confirm flow	43
7.4	Volunteer app – reporting confirmation, history, and profile	44
7.5	Mobile workflow, screens 1–3: Google sign-in	49
7.6	Mobile workflow, screens 4–6: home, alert detail, and live scan	50
7.7	Mobile workflow, screens 7–9: candidate match through confirmation	51
7.8	Mobile workflow, screens 10–12: post-match home and match history	52
7.9	Mobile workflow, screens 13–15: profile, personal info, and mesh screen	53
7.10	Mobile workflow, screens 16–18: mesh log, about, and sign-out	54
7.11	Mobile workflow, screen 19: session end – state restored to login	55
7.12	Offline mesh – live visualisation on the volunteer app	59

List of Tables

1.1	Literature and existing-system review	2
2.1	Functional requirements and their build/verification status	7
2.2	Non-functional requirements and their measurement status	8
2.3	Hardware resources used for development, testing, and demonstration	9
2.4	Software stack by layer	10
3.1	User actors and their interaction with the system	13
3.2	Core system modules and their responsibility	14
3.3	Lines-of-code count by component	14
4.1	Commit authorship	16
4.2	Representative commits, in order (full history: <code>git log on main</code>)	17
4.3	Weekly guide-assessment log, Week 1–10 (5 July – 15 September 2026), by member responsibility (Table 4.4)	19
4.4	Task ownership and expected contribution, as filed in the project proposal	22
4.5	Project phase schedule	23
5.1	Risk register	30
5.2	Mitigation strategy per identified risk	32
5.3	Risk projection – probability $\times$ impact	33
6.1	Languages, frameworks, and data store by component	35
6.2	Minimum and recommended hardware	36
7.1	<code>alerts/{alertId}</code> – the missing-child case record	37
7.2	<code>matches/{matchId}</code> – a volunteer-confirmed sighting	38
7.3	Kiosk data API surface (<code>lib/firestore.ts</code> , <code>lib/officers.ts</code>)	40
7.4	Threshold measurement – 36 labelled real pairs, original unaligned MobileFaceNet	46
7.5	Geometry parity – same photo, same identity, different alignment pipelines	46
7.6	Cloud Function exports and their trigger	56
7.7	Mesh packet wire format	58
7.8	Unit test suite – volunteer Android app	62
7.9	System-level test cases, executed on real hardware	63

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package Kit
BLE	Bluetooth Low Energy
CNN	Convolutional Neural Network
COCOMO	Constructive Cost Model
DFD	Data Flow Diagram
FCM	Firebase Cloud Messaging
FR	Functional Requirement
GPS	Global Positioning System
HMAC	Hash-based Message Authentication Code
IEEE	Institute of Electrical and Electronics Engineers
JDK	Java Development Kit
KLOC	Kilo Lines of Code
LFW	Labelled Faces in the Wild (benchmark dataset)
ML	Machine Learning
NCC	National Cadet Corps
NCRB	National Crime Records Bureau
NFR	Non-Functional Requirement
NGO	Non-Governmental Organisation
NSS	National Service Scheme
PWA	Progressive Web Application
RAM	Random Access Memory
SDK	Software Development Kit
SOLID	Single responsibility, Open–closed, Liskov substitution, Interface segregation, Dependency inversion
SPPU	(Affiliating) University – see title page
SRS	Software Requirement Specification
TFLite	TensorFlow Lite
TTL	Time To Live (hop-count field)
UI/UX	User Interface / User Experience
UUID	Universally Unique Identifier
WBS	Work Breakdown Structure

1 Introduction

1.1 Problem Definition

India reported **98,375** missing children in 2024 alone (National Crime Records Bureau), an average of **269 children every single day** [1]. NCRB's 2022 data records 83,350 reported cases – 20,380 males, 62,946 females, and 24 transgender minors – and a cumulative backlog of over 47,000 children who remain untraced at any given time, 71.4% of them minor girls. Despite a national recovery rate that hovers around 53–67% depending on the year and the measure used, recovery for the remainder routinely takes months or years rather than hours, leaving children exposed to trafficking, forced labour, and exploitation for every day they remain missing [2].

Both police officials and child-rights researchers converge on the same diagnosis: the **first 60–90 minutes** after a child is separated from a guardian – the *golden hour* – are the most critical window for recovery, and it is precisely this window in which the child is most at risk of being drawn into begging, trafficking, or criminal networks. This makes the golden hour a genuine race-against-time problem, and it is exactly the window in which India's large mass gatherings – Kumbh Mela, Dussehra, Ganesh Chaturthi visarjan processions, urban temple fairs – place the greatest strain on the two things a golden-hour response actually needs: present, coordinated human searchers, and a communication channel that still works.

Why existing systems fail at exactly the moment they are needed

A structural review of the systems currently in use or piloted in India (see the Literature Review, §1.2) surfaces four repeating limitations:

- (i) **Reactive, not proactive.** Systems such as TrackChild [2], Khoya-Paya [3], and ReUnite [4] operate on an “upload-first” basis: a photo must be found and uploaded *after* the fact, rather than the search actively scanning the crowd the moment a child is reported missing.
- (ii) **Centralised dependency.** Cloud-based face matching (e.g. ReUnite's [4] use of Amazon Rekognition) requires every comparison to round-trip to a remote server, introducing latency and a single point of failure.
- (iii) **Internet-dependent by construction.** Every reviewed system fails exactly when cellular towers saturate – which is the norm, not the exception, at India's largest festivals.
- (iv) **Closed to volunteers.** Police-only apps (Project Guardian [7]) or fixed, centre-bound kiosks (Digital Khoya-Paya Kendras [6]) cannot leverage the thousands of credible on-ground volunteers – NCC/NSS cadets, security staff, shopkeepers – physically present in the crowd.

A single police Lost & Found kiosk can manually scan only 50–100 faces per hour. In a festival crowd exceeding 100,000 people, this manual bottleneck alone makes the golden hour effectively unreachable through the operational model most Indian mass gatherings still rely on. At the same time, cloud-based facial-recognition alternatives raise a second, orthogonal problem: uploading a bystander’s biometric data to a central server, even transiently, is a privacy exposure that most festival infrastructure – lacking reliable connectivity to begin with – is not equipped to justify or secure.

NextGen Rakshak is built to close this specific, compound gap: give every registered volunteer’s ordinary smartphone the ability to *proactively* scan the crowd for a missing child, entirely on-device, so that recognition needs no round-trip to a server and no bystander’s face is ever transmitted, and so that the search keeps working when the cellular network that every existing system depends on does not.

1.2 Literature Review / Relevant Theory

Table 1.1 summarises the ten systems, standards, and underlying technologies surveyed while defining this project’s approach, and the specific structural gap each one leaves open that NextGen Rakshak is designed to close.

Table 1.1: Literature and existing-system review

#	System / Source	Year	Key finding and identified gap
1	NCRB missing-children statistics (via ThePrint) [1]	2023	National reporting shows a rising trend and a ~67% recovery rate, but resolution typically spans months to years – no minute-level “golden hour” window relevant to a live crowd event.
2	TrackChild Portal, Ministry of Women & Child Development [2]	2012	Central government database for missing/found children with inter-state coordination. <i>Gap</i> : reactive, post-incident record entry only; fully internet-dependent; no real-time search.
3	Khoya-Paya Portal [3]	2015	Citizen-facing portal for reporting missing/found children. <i>Gap</i> : manual search only – no automated face matching, no instant push alerts.
4	ReUnite – Bachpan Bachao Andolan + Capgemini (Amazon Rekognition) [4]	2018	Mobile app using cloud-based facial recognition to match uploaded photos. <i>Gap</i> : requires active internet and centralised cloud processing (privacy concern); reactive “upload-and-wait” model, not proactive scanning.

continued on next page

Table 1.1 continued from previous page

#	System / Source	Year	Key finding and identified gap
5	Garuda Rakshak – DSP Mutual Fund + Falco Robotics [5]	2025	Drone-based system using ID wristbands and real-time geo-data to locate separated children at Kumbh. <i>Gap</i> : requires physical wristband issuance and drone infrastructure per event; not scalable to smaller, everyday festivals.
6	Digital Khoya-Paya Kendra, UP Govt. (Maha Kumbh 2025) [6]	2025	10 AI-enabled fixed centres with facial recognition; ~54,357 people reunited, including all 18 reported-missing children. <i>Gap</i> : centralised and centre-bound – a person must physically reach a kendra; no push of the search into the moving crowd via volunteer devices.
7	Project Guardian, Thoothukudi District Police [7]	2025	Police-only smartphone app for instant photo sharing across duty officers; 12 children rescued at Dasara festival. <i>Gap</i> : restricted to police devices only; no automated face detection/recognition; no offline mesh capability.
8	AMBER Alert System, US DOJ [8]	2025	National wireless/broadcast alert system; 1,312 child recoveries, 252 via Wireless Emergency Alerts. <i>Gap</i> : network/cellular-infrastructure dependent; passive one-way broadcast; issued for under 1% of missing-child cases meeting strict abduction criteria.
9	MobileFaceNet [9]	2018	Lightweight CNN (<1M parameters), >99% accuracy on the LFW benchmark [15], 128-d embeddings, designed for on-device inference. <i>Gap</i> : it is a recognition <i>model</i> , not a deployable system – requires a detection pipeline, offline transport, and alert logic built around it.
10	Google Nearby Connections API [10]	2015	Peer-to-peer discovery and data transfer between nearby Android devices via BLE and Wi-Fi Direct, fully offline. <i>Gap</i> : provides only pairwise/cluster connections, not multi-hop mesh routing – reaching a volunteer several hops away requires a custom store-and-forward layer.

The resulting research gap

No single reviewed system combines all of the following in one architecture: (a) instant, decentralised push of a missing child’s photograph and face embedding to every registered volunteer within a geofenced radius, the moment an alert is filed; (b) fully on-device face matching, so that zero biometric data ever leaves the searching device; and (c) a genuinely offline fallback – a custom multi-hop store-and-forward routing layer built on Nearby Connections – so

alerts keep propagating hop-by-hop even when cellular networks are fully saturated. NextGen Rakshak's contribution is combining these three properties, which no reviewed solution offers together, around a realistic volunteer model of pre-registered, duty-bound participants rather than assuming spontaneous cooperation from anonymous festival attendees.

1.3 Scope

In scope

- On-device face detection (Google ML Kit) and recognition (MobileFaceNet / TensorFlow Lite) on volunteer Android smartphones.
- A police kiosk web application (Next.js) for filing, tracking, and resolving alerts, and reviewing volunteer-reported sightings.
- Geofenced push notification (Firebase Cloud Messaging) of alerts to registered volunteers within an approximately 2 km radius of the reporting kiosk.
- A fully offline, custom multi-hop store-and-forward mesh built on Google Nearby Connections (Bluetooth LE discovery, Wi-Fi Direct transfer) for alert and match propagation with no internet access at all.
- Human-in-the-loop confirmation: every candidate match must be visually confirmed by the volunteer before it is reported; the system never autonomously declares a child found.
- An optional Raspberry Pi fixed-camera node at a bounded exit gate (implemented last, as a stretch goal only).

Out of scope / explicit limitations

- The system does **not** replace formal police investigation, First Information Report (FIR) procedures, or the judicial process.
- It is designed for **golden-hour recovery at a bounded event**, not as a long-term, open-ended missing-person search tool.
- The volunteer's camera activates only after an alert has been received and the scan screen is explicitly opened by the volunteer – there is **no continuous or passive crowd surveillance**.
- The system targets a **pre-registered, duty-bound volunteer network** (police, NCC/NSS cadets, private security staff, shopkeepers, NGO partners) – it does not assume or require spontaneous participation by anonymous festival attendees.

- Face-matching accuracy on children’s faces specifically, and system behaviour under extreme concurrent load (50+ simultaneous alerts), are identified as open measurement items rather than claimed results – see Chapter 9, Future Scope.

1.4 Objectives

1. To design and develop a hybrid, edge-based system for rapid lost-child recovery at mass gatherings, addressing the critical “golden hour” gap identified by child-rights researchers and law-enforcement practitioners.
2. To implement on-device face detection using Google ML Kit [12] and face recognition using MobileFaceNet [9] with TensorFlow Lite [13], achieving real-time performance on standard Android devices without any cloud round-trip for recognition.
3. To enable both internet-based push notification (via Firebase [14] Cloud Messaging) and a fully offline, custom multi-hop store-and-forward mesh (built on the Google Nearby Connections API [10]) for alert distribution, so the system keeps working when cellular networks are congested.
4. To develop a police kiosk web application using Next.js with Firebase integration for rapid alert creation, volunteer coordination, and real-time match-alert display.
5. To build a volunteer Android application using Kotlin, CameraX, ML Kit [12], and TensorFlow Lite [13] that enables real-time crowd scanning for multiple concurrently active missing-child alerts.
6. To validate the system’s effectiveness against existing government and NGO solutions and demonstrate a meaningful improvement in search capacity over manual, human-only methods.

2 Requirement Analysis

2.1 Requirement Specifications

Requirements were classified using the Kano-style normal / expected / exciting model, which distinguishes requirements a stakeholder explicitly asks for from requirements they assume without stating, and from requirements that create disproportionate satisfaction precisely because they were not expected.

2.1.1 Normal Requirements

Requirements explicitly stated by the problem brief and directly traceable to an objective in §1.4: alert creation with a photo and case details at the kiosk; on-device face detection and recognition on the volunteer app; push notification of new alerts to nearby volunteers; a side-by-side confirmation step before a match is reported; and a live dashboard for the filing officer to review sightings.

2.1.2 Expected Requirements

Requirements a reasonable stakeholder assumes even though they may not state them: role-based authentication separating officers from volunteers; real-time (sub few-second) synchronisation of alert and match state across devices; automatic geofencing so alerts do not spam volunteers far from the event; automatic alert expiry so a stale case does not linger; and deletion of the child's photograph and face embedding once a case is resolved.

2.1.3 Exciting Requirements

Requirements that exceed the stated brief and materially differentiate the system from every system in the literature review: a genuinely offline, multi-hop mesh network (not just a single Bluetooth hop) with cryptographically authenticated packets; gateway-aware match routing, so an offline volunteer's confirmed sighting is preferentially relayed toward a peer with internet access and acknowledged end-to-end; a live, animated mesh-network visualisation screen for the volunteer; and automatic, SIM-backed contact-number population with dual-SIM picker support, removing a manual data-entry step that would otherwise slow volunteer onboarding during an event.

2.2 Validation of Requirements

Each requirement below is traceable to one or more of the six objectives in §1.4 and to a concrete, testable acceptance condition. Validation was carried out through (a) weekly guide review against the requirement list, (b) a measured face-matching threshold derived from real device data rather than assumed from literature (§7.3.3 discusses this measurement), and (c) an end-to-end walkthrough on physical Android hardware exercising the full alert-to-dispatch path, including the offline-mesh path across multiple physical devices. Requirements found to be unverifiable within the project timeline (e.g. recognition accuracy specifically on children's faces, or behaviour under 50 concurrent alerts) are explicitly flagged as open in Table 2.2 rather than reported as met.

2.3 Functional Requirements

Table 2.1: Functional requirements and their build/verification status

ID	Requirement	Status
FR-01	Officer can file a missing-child alert with a photo and structured case details from the kiosk	Built & verified
FR-02	A face embedding is computed for every alert and stored against the case	Built & verified (on-device primary, server-side fallback)
FR-03	New alerts are pushed to volunteers within a geofenced radius	Built & verified end-to-end
FR-04	Real-time on-device face detection with a frontality gate (reject extreme head-pose)	Built & verified on real device
FR-05	On-device face embedding extraction (TensorFlow Lite / LiteRT)	Built & verified
FR-06	Cosine similarity comparison of every detected face against all active alerts	Built & verified (live match observed, ~77% confidence)
FR-07	Side-by-side visual confirmation dialog, including an offline thumbnail fallback	Built & verified
FR-08	Volunteer explicitly confirms or dismisses a candidate match	Built & verified
FR-09	Confirmed match is reported to the kiosk with GPS location	Built & verified

ID	Requirement	Status
FR-10	Multi-hop mesh relay of alerts and matches with no internet available	Built; unit-tested and 2-device field trial complete; ≥ 3 -device field trial open
FR-11	System supports multiple simultaneous active alerts	Built; not yet load-tested at target scale (50)
FR-12	Alerts auto-expire after a configurable period (8 hours)	Built; scheduled sweep deployed, not timed end-to-end
FR-13	Optional Raspberry Pi fixed-camera gate node	Placeholder only – de-scoped unless time allows
FR-14	Every mesh packet carries a message ID and TTL, and is relayed at most once	Built & unit-tested

2.4 Non-Functional Requirement

Table 2.2: Non-functional requirements and their measurement status

ID	Requirement	State
NFR-01	Face detection accuracy $\geq 95\%$ on frontal faces	Not yet formally measured
NFR-02	Face recognition accuracy $\geq 90\%$ under festival-like lighting	100% observed on 36 adult test pairs; unproven for children / low light
NFR-03	Per-face inference latency < 200 ms on a mid-range Android device	Not yet formally timed
NFR-04	Alert delivery time < 5 s over the internet path	Observed fast in field use; not formally measured
NFR-05	Alert delivery time < 30 s over the offline mesh	Needs the ≥ 3 -device field trial
NFR-06	Volunteer scan throughput ≥ 200 faces/hour	Not yet measured
NFR-07	System supports 50 concurrent active alerts with no measurable performance loss	Not yet measured

ID	Requirement	State
NFR-08	Zero biometric data transmitted off-device (privacy compliance)	Enforced in code; no capture-based audit evidence write-up yet
NFR-09	Camera active only during an explicit scan session	Implemented; battery impact not measured
NFR-10	Runs on Android 5.0+	Superseded – project ships at minSdk 24 (Android 7.0+)

The threshold in NFR-02/NFR-04/NFR-05 is deliberately reported against measured evidence rather than an assumed pass, consistent with the project’s guiding principle of re-measuring rather than trusting a literature-derived default (see §7.3.3).

2.5 System Requirements

2.5.1 Hardware Requirements

Table 2.3: Hardware resources used for development, testing, and demonstration

Item	Qty.	Purpose
Android smartphone, 2 GB+ RAM, Android 8.0+	5	Volunteer app testing, offline multi-hop mesh trials
Laptop / tablet	1	Police kiosk portal test device
Development workstation (8 GB+ RAM)	–	Android Studio, Node.js/Next.js build, model conversion
Raspberry Pi 4 (4 GB) + Camera Module V2	1	Fixed-camera edge node at a simulated exit gate (optional extension)
USB power bank (5 V, 2.5 A)	1	Portable power for the Raspberry Pi node during a demo

2.5.2 Software Requirements

Table 2.4: Software stack by layer

Category	Tools / Technologies
Operating systems	Windows / Linux / macOS (development); Android 8.0+ (minSdk 24) runtime
Backend platform	Firebase: Firestore, Authentication, Cloud Storage, Cloud Functions (Node.js 22), Cloud Messaging
Web portal	Next.js 14, React 18, TypeScript, Tailwind CSS, shadcn/ui
Mobile app	Kotlin, Jetpack Compose, CameraX, Room, WorkManager
On-device ML	Google ML Kit (face detection), TensorFlow Lite / LiteRT (MobileFaceNet embedding)
Offline networking	Google Nearby Connections API (Play Services)
Model tooling	Python 3.10–3.12, TensorFlow, ONNX, NumPy (scripts/)
IDEs & tools	Android Studio, Visual Studio Code, Git & GitHub, Postman, Figma

3 System Model

3.1 Process Model

3.1.1 Incremental Model

NextGen Rakshak was developed using the **Incremental Process Model** [17]. The system was built as four functional increments, each adding a working, demonstrable capability on top of the previous one, rather than attempting a single “big bang” delivery at the end of the project:

- **Increment 1 – Core online path.** Police kiosk portal (alert creation, live dashboard), Firebase Authentication/Firestore schema, and the volunteer app skeleton with Google sign-in.
- **Increment 2 – On-device recognition.** ML Kit face detection with a frontality gate, 3-point similarity alignment, MobileFaceNet/TFLite embedding extraction, cosine-similarity matching, and the side-by-side confirmation UI.
- **Increment 3 – Offline mesh.** Nearby Connections discovery/transport, the custom store-and-forward routing layer (UUID message IDs, TTL, HMAC authentication, duplicate suppression), and gateway-aware match relay.
- **Increment 4 – Hardening and polish.** Multi-alert stress handling, landscape/tablet layouts, SIM-backed contact numbers, the live mesh-visualisation screen, dashboard analytics, and full system documentation.

3.1.2 Why to use Incremental Model

Three project-specific reasons drove this choice over a strict Waterfall or a full Agile/Scrum process:

1. **Four members, three independent codebases.** The web portal, Android app, and Cloud Functions can be built and tested in parallel once the Firestore schema (Increment 1) is fixed, which an incremental delivery plan naturally supports.
2. **Requirements changed based on measurement, not guesswork.** The face-match threshold moved from an assumed 0.75 (synopsis) to a measured 0.55 after real device testing, and authentication moved from anonymous sign-in to Google-only sign-in after a security review. An incremental model absorbs this kind of evidence-driven change between increments without invalidating work already done, unlike Waterfall.

3. **A demonstrable system had to exist early.** Weekly guide reviews required a working artefact, not just design documents – satisfied naturally once Increment 1 exists.

3.1.3 Advantages

- Working software is available after the very first increment, enabling early guide/stakeholder feedback.
- Risk is distributed across increments rather than concentrated at a single final integration step – e.g. the mesh-layer risk (R3, Chapter 5) was isolated to Increment 3 and did not block the already-working online path.
- Requirement changes discovered through testing (e.g. the threshold revision) can be absorbed in a later increment without a full project restart.
- Testing effort is spread throughout the project rather than compressed into one phase at the end.

3.1.4 Disadvantages

- Requires a well-defined overall architecture up front (§3.3) – an incorrectly scoped Increment 1 data model would have forced rework in every later increment.
- Total project cost/effort is harder to estimate at the outset than with Waterfall, since later increments may reveal work not visible during early planning (this occurred with mesh-layer packet authentication, added after Increment 3 began).
- Integration risk between increments (e.g. ensuring the mesh layer and the online Firestore path de-duplicate the same alert correctly) is higher than in a model where the whole system is integrated only once, by design, at the end.

3.2 System Breakdown Structure

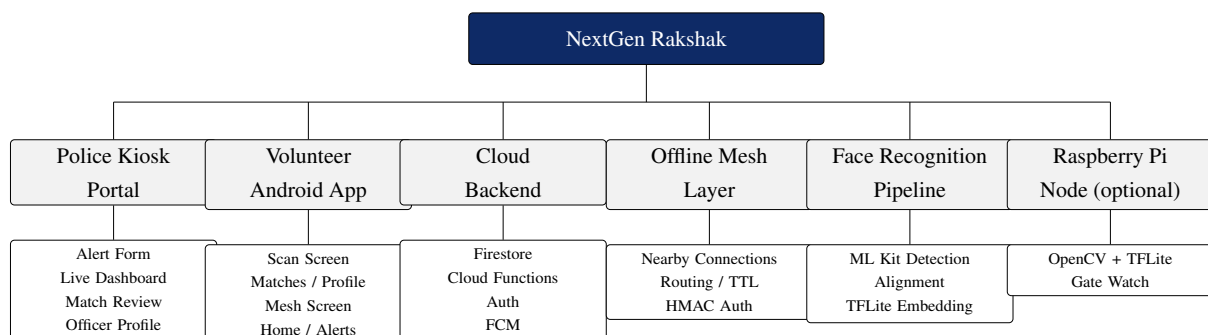


Figure 3.1: System breakdown structure

3.3 Module Details

3.3.1 User

Table 3.1: User actors and their interaction with the system

Actor	Role in the system
Police Officer	Signs in to the kiosk (Google sign-in → police custom claim); files, tracks, and resolves alerts; reviews and dispatches volunteer-reported sightings.
Volunteer (Police, NCC, NGO, Community)	Signs in with Google on the mobile app; receives geofenced alerts; scans the crowd; visually confirms or dismisses a candidate match; carries the offline mesh when out of network range.
System (automated)	Cloud Functions execute unattended: computing the fallback embedding, broadcasting the geofenced push, purging biometric data on resolve, and expiring stale alerts.

3.3.2 System Module

Table 3.2: Core system modules and their responsibility

Module	Responsibility
Alert Management	Create, list, filter, and resolve missing-child alert records; store the case photo and structured details.
Face Detection & Alignment	Locate faces in the live camera stream, reject non-frontal poses, and warp the face onto a canonical template before embedding.
Embedding & Matching Engine	Compute a fixed-length face embedding on-device and score it against every active alert with cosine similarity.
Notification & Geofencing	Compute which registered volunteers fall within the alert's geofence and dispatch a push notification through Firebase Cloud Messaging.
Offline Mesh Routing	Propagate alerts and confirmed sightings between devices with no internet, using TTL-bounded, authenticated, de-duplicated flooding.
Match Review & Dispatch	Present a volunteer's confirmed sighting to the filing officer side-by-side with the original photo, and let the officer accept, dismiss, or dispatch.
Data Lifecycle / Privacy Purge	Delete the case photo and clear the stored embedding the moment a case is resolved or expires.

3.4 Project Estimation

3.4.1 Estimation in KLOC

Project size was estimated using an actual line count of hand-authored source across the four components (auto-generated model schema files were excluded, as they are not project-authored code).

Table 3.3: Lines-of-code count by component

Component	Language	Files	LOC
Volunteer Android app	Kotlin	91	11,251
Police kiosk web portal	TypeScript/TSX	49	4,391
Cloud Functions	TypeScript	4	726
Model tooling & evaluation	Python	9	1,662
Total		153	18,030

Total size $\approx$ **18.0 KLOC**.

3.4.2 Efforts

Applying the **Basic COCOMO** model [16] in *Organic* mode (small, experienced team, a well-understood problem domain – appropriate for a four-member student team building on documented platforms):

$$E = a \cdot (\text{KLOC})^b, \quad a = 2.4, b = 1.05$$

$$E = 2.4 \times (18.0)^{1.05} \approx 2.4 \times 20.85 \approx \mathbf{50.0 \text{ person-months}}$$

3.4.3 Development time in months

$$D = c \cdot (E)^d, \quad c = 2.5, d = 0.38$$

$$D = 2.5 \times (50.0)^{0.38} \approx 2.5 \times 4.42 \approx \mathbf{11.1 \text{ months}}$$

3.4.4 Total Time Required for Project Development

COCOMO's organic-mode estimate of ≈ 11 months reflects a professionally staffed, production-hardened build. The team delivered a feature-complete MVP against the objectives in §1.4 within a compressed academic timeline of approximately 8 weeks of active development (Phases 1–9, Table 4.5), scoped deliberately as a validated prototype rather than a production-hardened system – the gap between the two estimates is consistent with, and explained by, the reduced scope (no formal QA cycle, a single target platform, and a small, pre-defined feature set) appropriate to a final-year academic deliverable.

3.4.5 Number of Developers (N)

$$N = \frac{E}{D} = \frac{50.0}{11.1} \approx \mathbf{4.5 \text{ people}}$$

This COCOMO-derived average staffing figure of ≈ 4 –5 people aligns closely with the actual team size of four, supporting the estimation model's applicability to this project despite the compressed academic schedule.

4 System Analysis

4.1 Project Scheduling and Tracking

4.1.1 Project Work and Breakdown Structure (Analysis)

The analysis phase (Phases 1–2 of the project schedule) decomposed the problem into requirement gathering and system design work packages before any code was written.

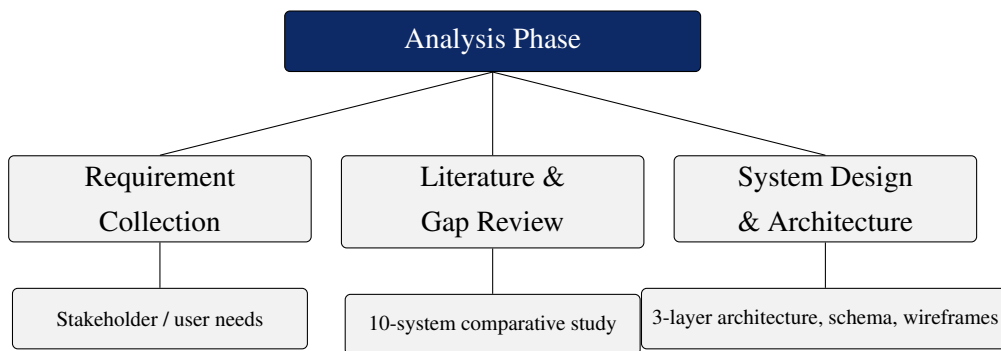


Figure 4.1: Work breakdown structure – Analysis phase

4.1.2 Version Control and Commit History

The entire project – all three components, the Cloud Functions, the model scripts, and every weekly report – is tracked in a single public Git repository hosted on GitHub at <https://github.com/ByteNinjaSmit/NextGen-Rakshak>. The commit history is the project’s actual, unedited build log: 64 commits from the first commit on 21 July 2026 to the last on 13 September 2026, an unbroken ~8-week record of the incremental process described in §3.1.

Table 4.1: Commit authorship

Author	Commits	Track (Table 4.4)
Smitraj Bankar (ByteNinjaSmit)	53	Architecture, ML pipeline, mesh, Cloud Functions, integration
AtharvaNarkhede1	10	UI/UX, web portal styling, integration testing
Vedant Dhadge	1	Android/Firebase integration

Two feature branches, AndriodUI and WebPortalUI, were split off main for the Week 5–6 UI overhaul and merged back through reviewed pull requests (PR #1 merged WebPortalUI on

12 August 2026) – the branch discipline identified as a gap in the Week 4 risk review (R10, §5.1) was adopted from that point on, rather than left as an unaddressed finding. Commit subjects follow the Conventional Commits convention (feat:, fix:, docs:, build:, chore:), which is what makes the table below possible to produce directly from `git log` rather than from memory.

Table 4.2: Representative commits, in order (full history: `git log on main`)

Date	Hash	Subject
21 Jul	2211d78	feat: initial commit – NextGen Rakshak lost-child recovery system
21 Jul	022ff83	feat: side-by-side match confirmation, match haptics, photo purge on resolve
22 Jul	e727b69	feat: working MobileFaceNet pipeline – SavedModel on server, tflite on device
22 Jul	f9128a9	feat: lower match threshold to 0.55 from measurement, add Week 3 documentation
22 Jul	a96c7b3	fix: gate the police kiosk on a police claim, not merely being signed in
22 Jul	998307b	build: switch to LiteRT and Kotlin 2.3 so all 64-bit libraries are 16 KB aligned
29 Jul	f80f30e	feat: self-register kiosk officers, smooth login/logout popup UX
30 Jul	0f40a9d	feat: mesh case-closure, hardened match writes, offline fixes
01 Aug	b5a6f57	feat: dashboard active alerts/live match update, new alert form fields
12 Aug	67cb647	Merge pull request #1 from ByteNinjaSmit/WebPortalUI
13 Aug	9e4fc3f	feat: kiosk notification bell with FCM match push
13 Aug	136a28b	chore: upgrade functions to Node 22 and firebase-functions v7
15 Aug	0d35a5c	feat: implement MatchesScreen, MatchConfirmation dialog, updated UI themes
23 Aug	d741609	android face done
02 Sep	ef3998f	feat: face-match accuracy – alignment, multi-frame fusion, quality gate
02 Sep	8a3dc4a	feat: offline mesh – advanced pass (thumbnail, HMAC, foreground service, persistence)

Date	Hash	Subject
02 Sep	4d2f9a8	fix: location & geofencing review – rules regression + fail-open hardening
05 Sep	f ad33c5	feat: simplify auth to Google-only, enforce system roles, camera switch
07 Sep	bb0f829	fix: restore live face matching – model deadlock, geometry drift, track pollution
13 Sep	8a3d32a	feat(mobile): SIM-backed contact number, Google avatar, live mesh screen
13 Sep	dd37ec8	feat(webportal): interactive dashboard stats, pagination/filters, layout scroll fix
13 Sep	33062cf	fix: fall back to placeholder icon when alert photo fails to load

The trajectory the log records matches the incremental plan exactly rather than being reconstructed after the fact: the online path (kiosk, auth, first face pipeline) lands in the first two weeks; the threshold revision from an assumed 0.75 to a measured 0.55 (f9128a9) and the switch from anonymous to Google sign-in (9e73b85) are both mid-project corrections driven by evidence, not planning; the mesh layer’s authentication, thumbnail, and gateway-routing work (8a3dc4a, 6aed293) lands in a single dense week once the online path was stable; and the final fortnight (bb0f829 onward) is entirely defect-fix and polish commits against real-device testing, exactly as the Week 7–8 schedule in Table 4.5 intended.

4.1.3 Weekly Progress Log (Guide-Assessed)

Independently of the phase table above, the department’s weekly assessment process tracked the team against a 10-week guide-reviewed log, one signed form per week (reproduced in full, with each week’s guide remark, in docs/weekly-reports/). **Both records agree on the same conclusion: the system itself – both applications, the cloud backend, and the offline mesh – was fully built, integrated, and tested within these 10 weeks, with final integration complete by 15 September 2026.** Table 4.5’s later phases (research-paper drafting, copyright filing) are post-completion academic output, not outstanding system work; they are tracked separately in §9.2.

Table 4.3: Weekly guide-assessment log, Week 1–10 (5 July – 15 September 2026), by member responsibility (Table 4.4)

Wk	Dates	Work done, by member
1	05–11 Jul	Bankar: reviewed MobileFaceNet/TFLite and ML Kit documentation for on-device recognition feasibility. Bhakare: reviewed TrackChild, Khoya-Paya, and ReUnite, establishing the reactive/server-dependent research gap. Narkhede: compiled the NCRB missing-children figures and the Supreme Court’s golden-hour observation to frame the problem. Dhadge: studied the Nearby Connections API for offline peer-to-peer feasibility.
2	12–18 Jul	Bankar: drafted the three-layer (Cloud/Edge/Mesh) architecture diagram and finalised the technology stack. Bhakare: designed the inter-component interaction flow (kiosk → FCM → volunteer → mesh) and communication protocols. Narkhede: designed UI wireframes for the kiosk and Android app. Dhadge: planned the Firestore schema (alerts, volunteers, matches, roles).
3	19–25 Jul	Bankar: studied Firebase Cloud Functions for embedding computation and alert-broadcast logic. Bhakare: studied MobileFaceNet/TFLite implementation and model validation (quantisation, accuracy evaluation). Narkhede: designed UI wireframes for the kiosk and Android app screens. Dhadge: studied Firebase Authentication and role-based access control.
4	26 Jul–01 Aug	Bankar: tested Cloud Functions and Firestore operations; deployed the kiosk to Vercel for iterative testing. Bhakare: studied the Next.js framework – routing, project structure, and setup. Narkhede: implemented frontend UI colour changes, dashboard alerts/live-match updates, the new alert form, and login/register pages. Dhadge: studied Android fundamentals, set up the dev environment, and scaffolded the bare-bones volunteer app.

Wk	Dates	Work done, by member
5	02–08 Aug	Bankar: implemented Cloud Functions (embedding, FCM broadcast, match processing) with Firestore triggers; deployed and verified. Bhakare: studied Next.js App/API routes, server/client components, and middleware; reviewed the implemented pages. Narkhede: completed the dashboard UI/UX with Firebase integration – login/register, live updates, profile, validation, responsive layout. Dhadge: implemented runtime permissions (camera, location, Nearby), the scan-screen UI, and navigation.
6	09–15 Aug	Bankar: redeployed Cloud Functions on the upgraded Node runtime; sourced MobileFaceNet weights and built the conversion scripts. Bhakare: measured recognition accuracy on 36 labelled pairs using the quantised model (see §7.3.3 for the figures carried into this report). Narkhede: ran the Android/web cross-platform consistency pass and UI language alignment. Dhadge: scoped the TPEB paper and tested the Android build on the UI branch.
7	16–22 Aug	Bankar: optimised Cloud Functions (cold-start, response time) and redeployed. Bhakare: finalised the MobileFaceNet .tflite integration, tuned the threshold against the measured pairs, and optimised for low-end devices. Narkhede: completed kiosk/Android sync, the role-based dashboard, and profile validation. Dhadge: implemented the offline mesh layer (Nearby Connections, multi-hop store-and-forward) with working peer discovery.
8	23–29 Aug	Bankar: ran backend load and Firestore-rules testing, and fixed FCM broadcast reliability issues. Bhakare: tested on-device recognition accuracy under varied lighting. Narkhede: completed full kiosk integration testing (auth, dashboard, alerts, profile, responsive/cross-browser) and fixed UI inconsistencies. Dhadge: ran Android instrumentation testing, the offline-mesh field test, and battery/network profiling.
9	30 Aug–05 Sep	Bankar: finalised the production deployment (Vercel + Firebase) with a Firestore backup and backend documentation. Bhakare: finalised the accuracy write-up, model optimisation, and inference benchmarking documentation. Narkhede: completed final UI/UX polish, the user manual, presentation slides, and demo-flow verification. Dhadge: built the final release APK, ran final device testing, and contributed to the TPEB paper draft.

Wk	Dates	Work done, by member
10	06–15 Sep	Bankar: diagnosed and fixed the two mesh reconnect-stability defects found in the 3-device field trial (§7.9.4) – the missing runtime NEARBY_WIFI_DEVICES request and the onEndpointLost miswiring (§7.6) – and the TFLite interpreter concurrency deadlock (§7.3.4). Bhakare: shipped the kiosk’s interactive dashboard – clickable stat tiles, match-status chart, top-volunteers leaderboard – with pagination/filters and a layout scroll fix. Narkhede: verified the field-trial fixes end-to-end and ran final UI/UX validation across the redesigned mesh and dashboard screens. Dhadge: shipped the SIM-backed volunteer contact number (dual-SIM picker), Google-avatar profile identity, and the live mesh-visualisation screen. System declared feature-complete and frozen on 15 September 2026 (Table 4.2).

Every item in Week 10 traces to a specific commit in Table 4.2 (bb0f829 through 33062cf); nothing in this final week is a desk-review finding – all of it, including the two mesh defects, surfaced only once the finished system was exercised end-to-end on real, physically separate devices, which is precisely why it took a dedicated closing week rather than being caught earlier.

4.2 Project Work Breakdown Structure (Implementation)

The implementation phase (Phases 3–9) decomposed into three parallel development sprints followed by integration, hardening, and deployment.

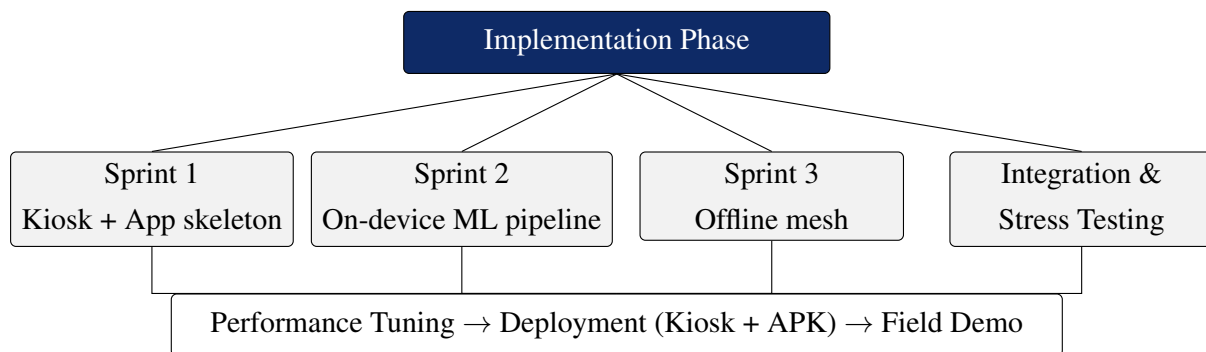


Figure 4.2: Work breakdown structure – Implementation phase

4.3 Task Identification

4.3.1 Ownership and Responsibility (as filed in the Project Proposal)

Task ownership was fixed at the proposal stage, before any code was written, and held for the life of the project – every week in the 10-week log (Table 4.3) is an instance of the same four-way

split defined here.

Table 4.4: Task ownership and expected contribution, as filed in the project proposal

Student	Responsibility	Contribution
Bankar Smitraj Dinkar (09)	Overall project lead and system architect – owns the end-to-end architecture (Cloud/Edge/Mesh layers), the backend (Firebase Cloud Functions, Firestore schema), on-device ML pipeline integration (ML Kit + MobileFaceNet/TFLite), the offline mesh implementation (Nearby Connections + custom routing), cross-component integration, and final report/research paper/patent-filing coordination.	~40% – First Author
Bhakare Tanishka Sharad (11)	Web portal development – builds the Police Kiosk Portal in Next.js/React/TypeScript/Tailwind CSS, including the alert-creation form, live matches dashboard, and Firebase integration (Auth, Firestore listeners).	~20% – Co-Author
Narkhede Atharva Anantkumar (94)	UI/UX design across both platforms and QA – designs wireframes (Figma) for the Police Kiosk Portal and Volunteer Android App interfaces, and leads functional/integration testing across the system (multi-device offline-mesh tests, multi-alert stress tests, UI/UX validation).	~20% – Co-Author
Dhadge Vedant Sanjay (34)	Android application development and Firebase integration – builds the Volunteer Android App (Kotlin, Jetpack Compose, CameraX) and wires it to Firebase (Authentication, Firestore sync, Cloud Messaging) for alert receipt and match reporting.	~20% – Co-Author

4.3.2 Project Schedule

Table 4.5 lists the twelve project phases as planned and tracked from the project’s governance record, spanning requirement collection through report writing, performance tuning, deployment, field demonstration, and copyright filing preparation.

4.4 Project Table and Time-line Chart

4.4.1 Project Schedule Time

Table 4.5: Project phase schedule

Phase	Milestone	Duration	Start
1	Requirement collection, literature review, synopsis finalisation	2 weeks	11 Aug 2026
2	System design: three-layer architecture, schema, API contracts, wireframes	2 weeks	12 Aug 2026
3	Dev Sprint 1: Police kiosk portal + volunteer app skeleton	2 weeks	13 Aug 2026
4	Dev Sprint 2: On-device ML pipeline (detection, embedding, matching)	2 weeks	27 Aug 2026
5	Dev Sprint 3: Offline mesh (routing, TTL, dedup) + optional Pi node	1 week	03 Sep 2026
6	Integration, multi-alert stress test, 5–10 device mesh test, bug-fix	1 week	10 Sep 2026
7	Final project report writing & documentation	2 weeks	24 Sep 2026
8	Performance optimisation: threshold calibration, latency, battery tuning	2 weeks	08 Oct 2026
9	Deployment: hosted kiosk, sideloaded volunteer APK, optional Pi field setup	2 weeks	22 Oct 2026
10	Field test / pilot demonstration with 5–10 volunteers	3 weeks	12 Nov 2026
11	Research-paper drafting from field-test and benchmark results	3 weeks	03 Dec 2026
12	Patent / copyright filing preparation and submission	4 weeks	15 Jan 2027

4.4.2 Time-Line Chart

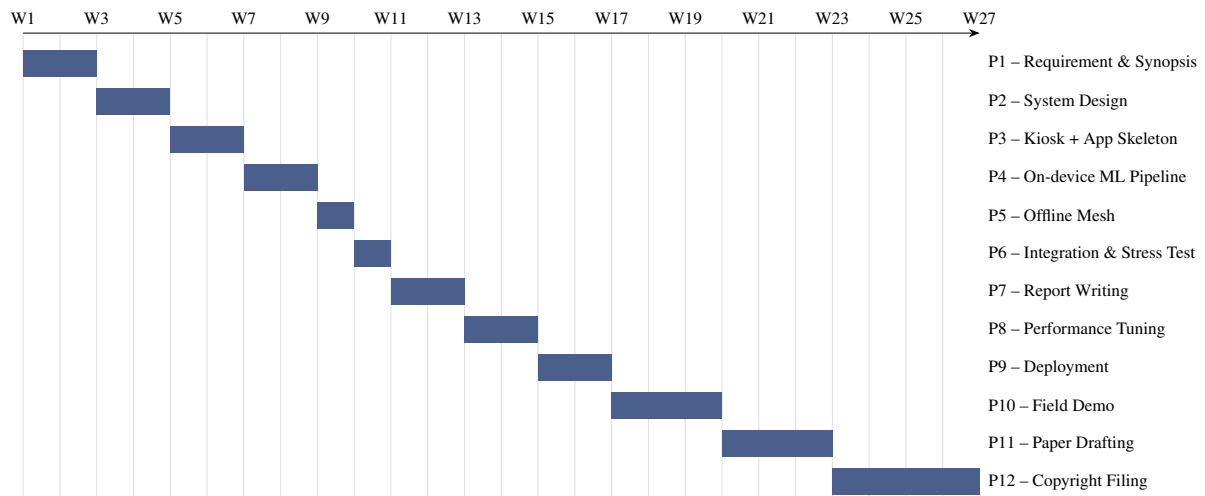


Figure 4.3: Project time-line chart (weeks from Phase 1 start, 11 Aug 2026)

4.5 Analysis Modeling

4.5.1 Behavioral Modeling

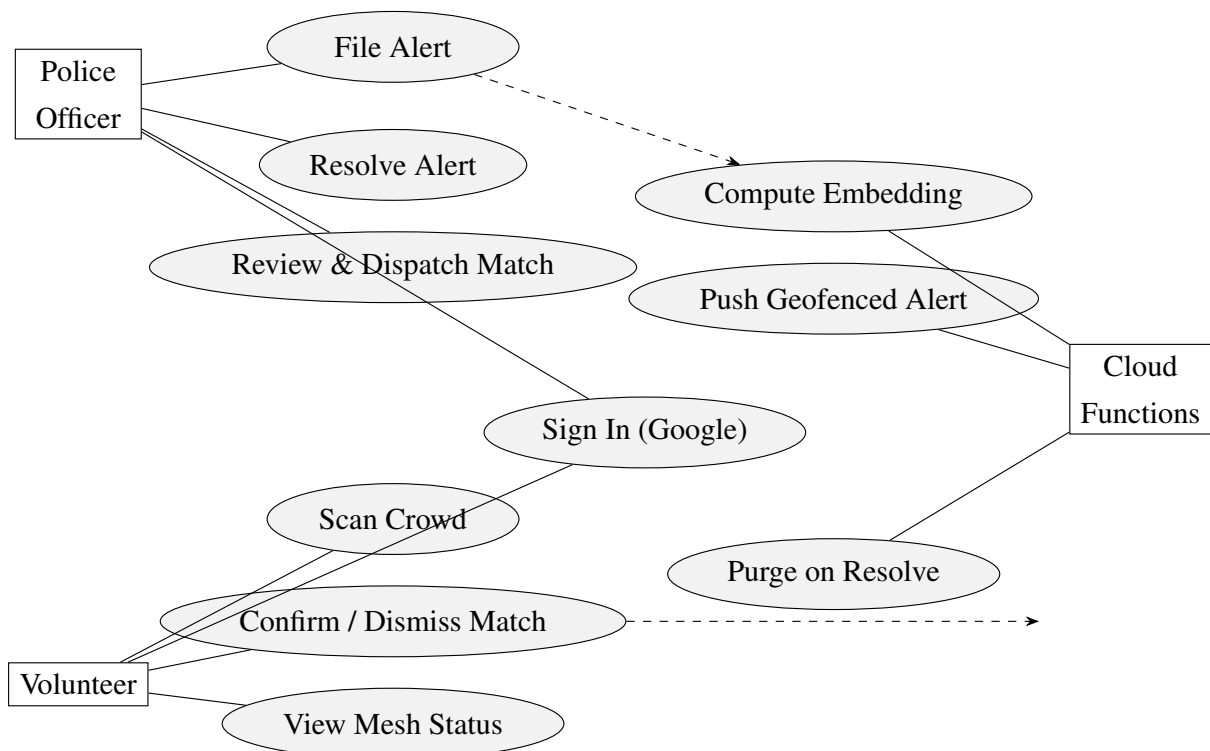


Figure 4.4: Use case diagram – Police Officer, Volunteer, and Cloud Functions

Use Case Diagram.

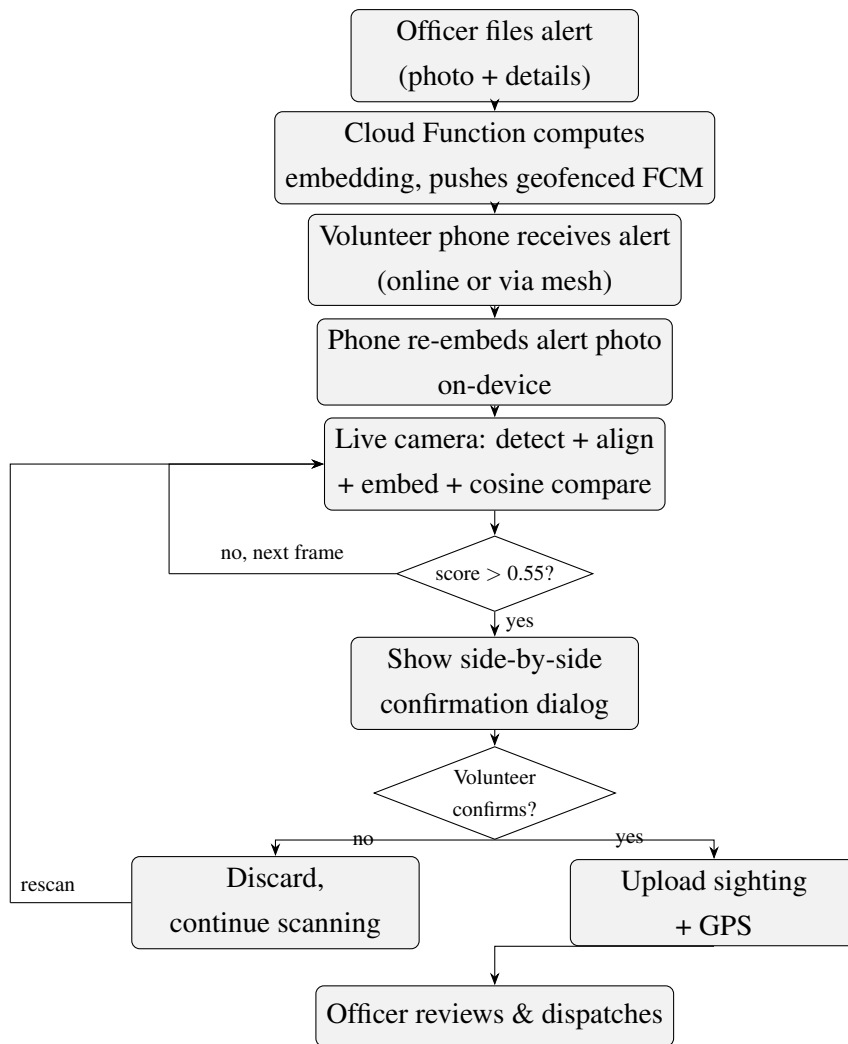


Figure 4.5: Activity diagram – alert filing through confirmed sighting

Activity Diagram – Alert-to-Match Flow.

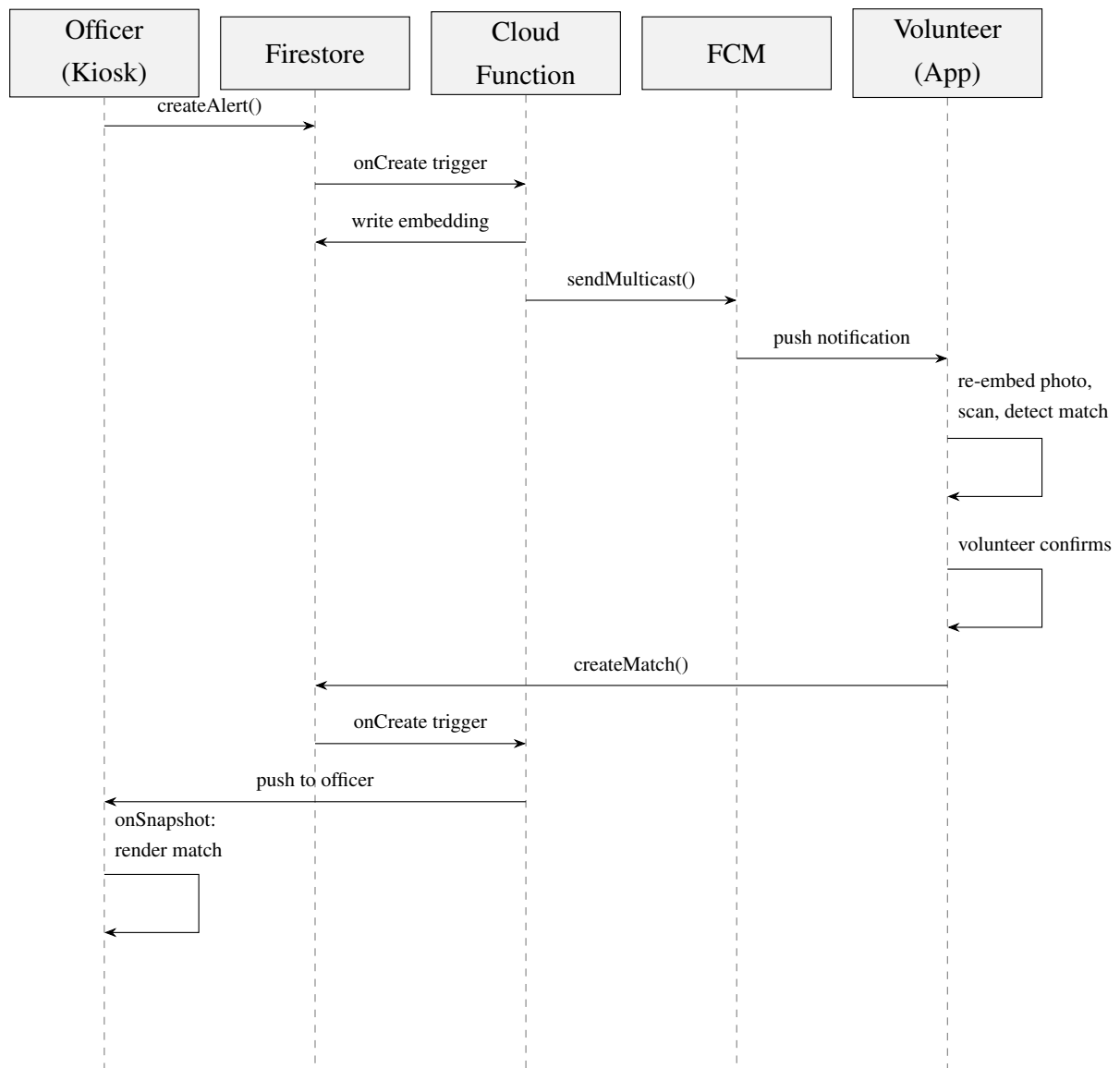


Figure 4.6: Sequence diagram – alert creation through match notification

Sequence Diagram – Confirmed Match (Online Path).

4.5.2 Functional Modeling

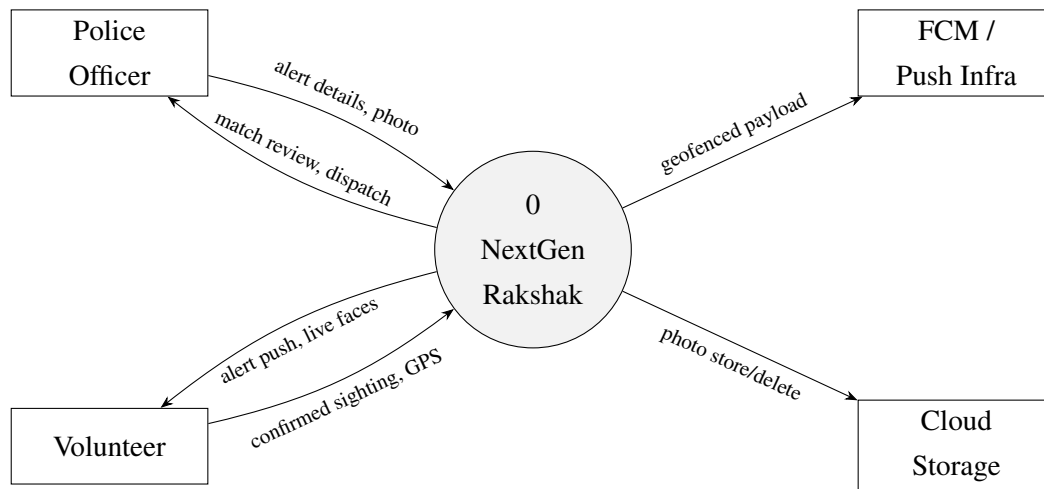


Figure 4.7: DFD Level 0 – system context

DFD Level 0 (Context Diagram).

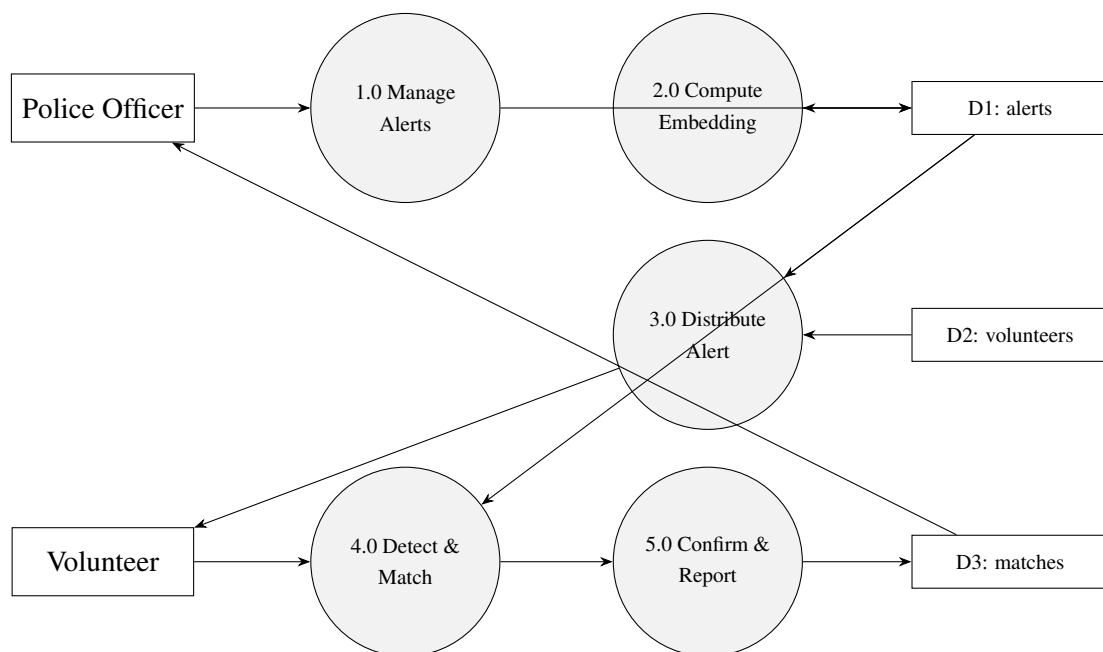


Figure 4.8: DFD Level 1 – decomposition of core processes

DFD Level 1.

4.5.3 Architectural Modeling

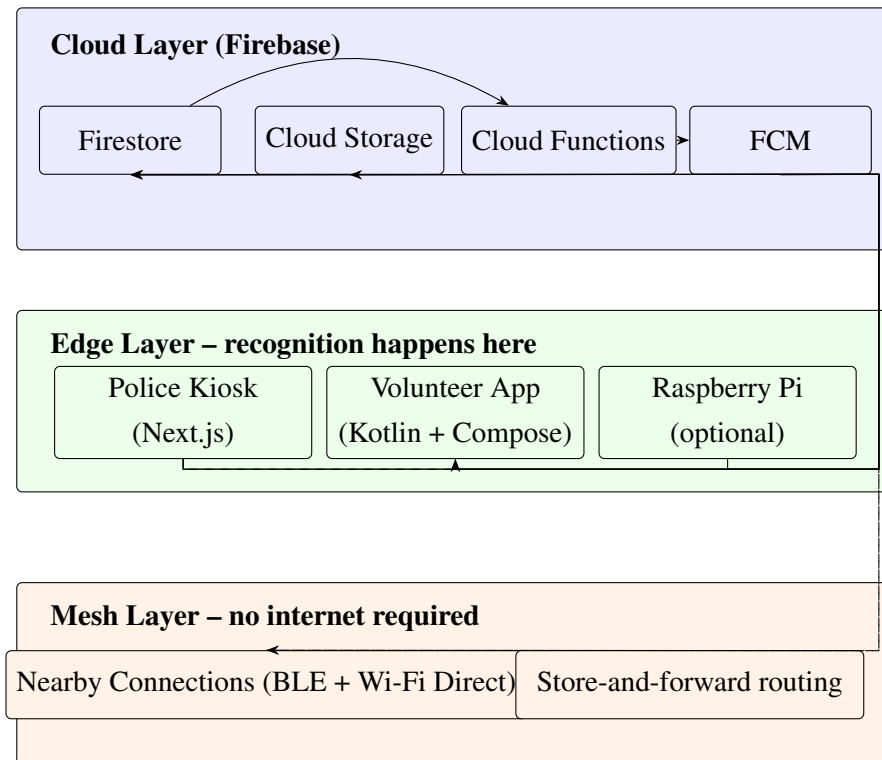


Figure 4.9: Three-layer system architecture: Cloud, Edge, Mesh

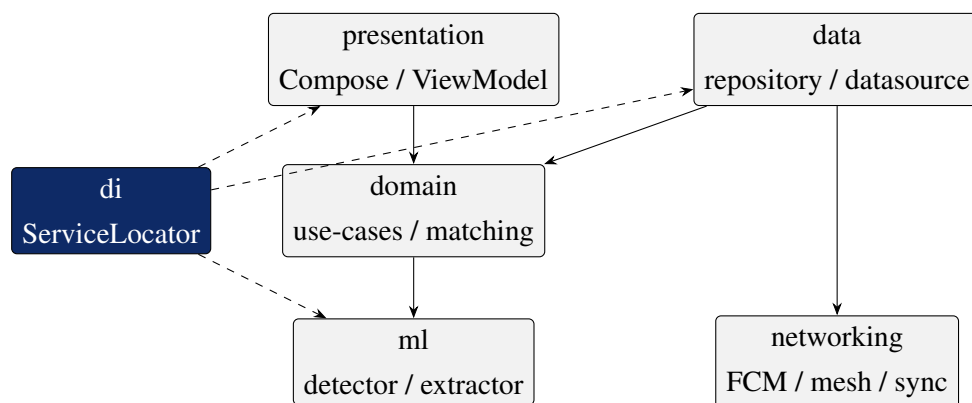


Figure 4.10: Volunteer Android app – layered (SOLID) package architecture

4.6 Mathematical Modeling

The system is formally represented as:

$$S = \{I, P, O, F_{\text{main}}, D_D, D_{\text{fail}}\}$$

where

$I = \{\text{alert photo, case details, live camera frames, GPS fix}\}$	(inputs)
$P = \{\text{detect, align, embed, compare, confirm, route, dispatch}\}$	(processes)
$O = \{\text{confirmed sighting, dispatch decision}\}$	(outputs)
$D_D = \text{successfully reported, GPS-tagged, officer-reviewed sighting}$	(success)
$D_{\text{fail}} = \{\text{no camera / no GPS / low light / occluded face}\}$	(failure)

Face matching itself is a function

$$\text{match}(e_1, e_2) = \begin{cases} 1 & \text{if } \cos \theta = \frac{e_1 \cdot e_2}{\|e_1\| \|e_2\|} > \tau \\ 0 & \text{otherwise} \end{cases}$$

where $e_1, e_2 \in \mathbb{R}^{128}$ are L2-normalised face embeddings and $\tau = 0.55$ is the measured decision threshold (§7.3.3). Because both e_1 and e_2 are unit vectors, cosine similarity reduces to their dot product, making the comparison a single, cheap floating-point operation suitable for repeated per-frame execution on a mobile CPU.

4.6.1 Venn Diagram

Figure 4.11 formalises the research gap already identified qualitatively in Table 1.1: every reviewed existing system falls inside at least one of three structural-limitation sets – *Centralised*, *Internet-dependent*, and *Reactive* – whereas NextGen Rakshak is positioned outside all three.

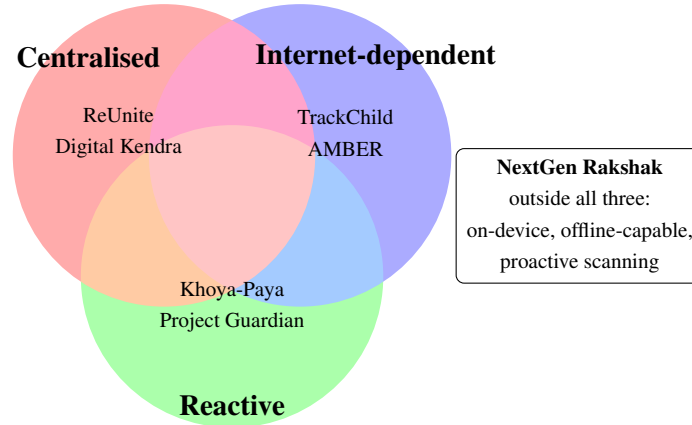


Figure 4.11: Venn diagram – structural limitations of existing systems vs. NextGen Rakshak's position

5 Risk Management

5.1 Risk Identification

Ten risks were identified and tracked across the project, ranked by expected damage to the project rather than by probability alone.

Table 5.1: Risk register

ID	Risk	Impact	Owner
R1	Server-side ML runtime (tfjs-node) fails to load in the deployed Cloud Functions environment	Critical	Cloud/Functions owner
R2	No suitable, ethically sourced children's face dataset for recognition-accuracy validation	High	ML/evaluation owner
R3	The offline mesh has never been validated across real, physically separate devices	High	Mesh owner
R4	Final-report writing load exceeds what one person can produce in the time remaining	High	Report lead
R5	Feature freeze slips and features are still being added during report-writing weeks	High	Project lead
R6	Live demo fails on stage due to venue Wi-Fi, FCM latency, or device issues	High	All members
R7	Low automated test coverage leaves most code paths unverified by anything but manual testing	Medium	QA owner
R8	No officer is authorised, so the kiosk cannot be used by anyone until this is fixed	Medium	Auth owner

ID	Risk	Impact	Owner
R9	The offline demo path cannot render the child's photo without internet, since only a URL is carried by default	Medium	Mesh/UI owner
R10	Four people committing directly to main produces avoidable merge conflicts	Low– Medium	Project lead

5.2 Strategies Used to Manage Risk

Table 5.2: Mitigation strategy per identified risk

ID	Mitigation
R1	Deploy early rather than late, so a runtime failure is caught with time to react; fallback plan was to compute the embedding client-side with TensorFlow.js at alert creation instead of server-side.
R2	Ordered fallback: seek a licensed academic dataset first; else use team members' own consented childhood photographs; failing both, report the adult-only measurement plainly as an upper-bound proxy rather than a child-accuracy result, and list it as Future Scope.
R3	Budget a full day for a dedicated multi-device trial with verbose logging added <i>before</i> the trial, not after; report the maximum hop count actually observed rather than an unverified claim.
R4	Confirm early whether a conference paper is required (removes significant load if not); start results-chapter writing the moment the last measurement is available, not after.
R5	Enforce a hard feature-freeze date; anything unfinished at that point becomes a documented Future Scope item, not a late addition.
R6	Prepare a pre-recorded demo video, a personal-hotspot network backup, and printed screenshots; lead with the offline-mesh demo, since it is the scenario that needs no network at all.
R7	Raise unit-test coverage on the core matching/mesh logic and exercise every functional requirement at least once before submission.
R8	Fix officer self-registration (the <code>claimOfficerRole</code> function) early and verify both the grant path and the deny path explicitly.
R9	Carry a small, highly compressed JPEG thumbnail of the child's face inside the mesh packet itself, so the offline confirmation dialog can render a photo with zero internet.
R10	Adopt short-lived feature branches and pull requests once parallel development begins, rather than direct commits to <code>main</code> .

5.3 Risk Projection

5.3.1 Preparing Risk Table

Risks were re-scored qualitatively (Low / Medium / High) on probability and impact, and a priority derived from their product, to decide where limited remaining project time should go.

Table 5.3: Risk projection – probability $\times$ impact

ID	Risk	Probability	Impact	Priority
R1	tfjs-node runtime failure	Medium	Critical	Highest
R3	Mesh unverified on real devices	Medium	High	High
R2	No children’s face dataset	High	High	High
R6	Live demo failure	Medium	High	High
R5	Feature-freeze slip	Medium–High	High	High
R4	Report-writing overload	Medium–High	High	Medium–High
R8	No authorised officer	Certain (until fixed)	Medium	Medium
R9	Offline demo shows no photo	High (without fix)	Medium	Medium
R7	Low test coverage	Certain	Medium	Medium
R10	No branch discipline	Medium	Low–Medium	Low

5.4 Feasibility

Technical feasibility

Every component relies on mature, freely available, and well-documented technology: Google ML Kit [12] and TensorFlow Lite [13] ship with pretrained, production-grade models; Firebase [14] provides a realtime backend with no server for the team to operate; and Nearby Connections [10] is a maintained Google Play Services API rather than a custom radio protocol. The riskiest technical assumption – that a lightweight, on-device model could achieve useful discrimination – was validated directly through the threshold measurement in §7.3.3, rather than assumed. The project is therefore judged technically feasible for a four-member student team within one academic year.

Operational feasibility

The volunteer model targets pre-registered, duty-bound participants (police, NCC/NSS cadets, NGO staff, shopkeepers) already present at mass gatherings in an official or semi-official capacity, rather than requiring spontaneous adoption by the general public – this matches how Indian festival security is actually staffed today, and keeps onboarding to a single Google sign-in.

Economic feasibility

The entire stack sits within free or near-free tiers for a project of this scale: Firebase’s Spark/Blaze free-tier quotas comfortably cover a demonstration deployment; ML Kit, TensorFlow Lite, and Nearby Connections carry no licensing cost; and hardware requirements are ordinary Android phones the volunteer network already owns. The only new hardware is the optional Raspberry Pi node, budgeted at under 5,000 (Table 2.5.1) and explicitly scoped as optional.

Schedule feasibility

The twelve-phase schedule (Table 4.5) allocates dedicated weeks to integration and stress testing (Phase 6), performance tuning (Phase 8), and a field demonstration (Phase 10) before report and paper deadlines, rather than compressing verification into the final days – directly addressing the schedule risks (R4, R5) identified in §5.1.

6 Technical Specification

6.1 Software requirement specification

6.1.1 Operating System (OS)

Development was carried out across Windows, Linux, and macOS workstations (the toolchain – Node.js, Android Studio/Gradle, Python – is cross-platform). The volunteer application targets Android `minSdk 24` (Android 7.0) through `compileSdk 35` (Android 15); the kiosk portal runs in any modern evergreen browser (Chrome, Edge, Firefox) on the police booth’s existing device, requiring no dedicated OS.

6.1.2 Integrated Development Environment (IDE)

- **Android Studio** – volunteer app development, Gradle builds, on-device debugging and the LayoutInspector for the Compose UI.
- **Visual Studio Code** – Next.js/TypeScript kiosk portal and Cloud Functions development.
- **Figma** – UI/UX wireframing for both the kiosk and mobile screens.
- **Postman** – manual exercising of Cloud Function callable endpoints during development.

6.1.3 Programming Language and Database

Table 6.1: Languages, frameworks, and data store by component

Component	Language / Framework
Police kiosk portal	TypeScript, Next.js 14 (App Router), React 18, Tailwind CSS, shadcn/ui
Volunteer Android app	Kotlin, Jetpack Compose, Coroutines, Room (SQLite), DataStore
Cloud Functions	TypeScript, Node.js 22, firebase-functions v7, @tensorflow/tfjs-node
Model tooling	Python 3.10–3.12, TensorFlow, ONNX
Database	Cloud Firestore (NoSQL document store) – collections alerts, matches, officers, volunteers; on-device Room (SQLite) for the offline match/mesh queue

6.2 Hardware Requirement Specifications

Table 6.2: Minimum and recommended hardware

Device	Minimum	Recommended
Volunteer phone	Android 7.0 (API 24), 2 GB RAM, rear camera, Bluetooth LE	Android 10+, 4 GB RAM, dual-SIM, GPS
Kiosk device	Any laptop/tablet with a modern browser	Laptop with a stable network connection
Development machine	8 GB RAM, JDK 17, Node.js 18+	16 GB RAM, SSD, Node.js 22
Raspberry Pi node (optional)	Pi 4 (4 GB) + Camera Module V2	Pi 4 (4 GB) + Camera Module V2 + active cooling

7 Implementation Details

This chapter documents how each module described in §3.3 and §4.5 was actually implemented, with representative code and screenshots of the working system.

7.1 Firestore Data Model

Every component reads and writes the same four Firestore collections. The schema is deliberately *denormalised* – a matches document copies `childName` and `imageUrl` from its parent alert – so that the kiosk’s live feed and the volunteer’s match history each render from a single collection listener, with no join and no second round-trip.

Table 7.1: `alerts/{alertId}` – the missing-child case record

Field	Type	Notes
<code>childName</code> , <code>age</code> , <code>gender</code>	string, number, string	Displayed on every volunteer device
<code>clothingDesc</code> , <code>identifyingMarks</code>	string	Human search aid; usable even if recognition fails
<code>parentContact</code>	string	Kiosk-only; never sent to volunteer devices or the mesh
<code>imageUrl</code>	string	Cloud Storage URL; blanked when the case resolves
<code>embedding</code>	number[]	128-d fallback vector; written by <code>onAlertCreated</code> ; cleared on resolve
<code>lastSeen</code> , <code>lastSeenDate/Time</code>	string	Free-text place plus a structured date/time
<code>geoLocation</code>	GeoPoint?	Kiosk’s fix at filing; drives the 2 km geofence
<code>status</code>	string	<code>active</code> <code>resolved</code>
<code>timestamp</code>	Timestamp	Server clock; drives expiry and elapsed-time display

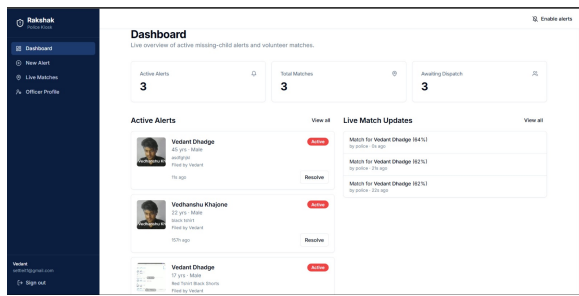
Table 7.2: matches/{matchId} – a volunteer-confirmed sighting

Field	Type	Notes
alertId	string	Rules require the referenced alert to exist
childName, imageUrl	string	Denormalised for single-listener rendering
volunteerId, volunteerName	string	Reporter’s Auth uid and display name
relayedBy	string?	Set only when a <i>different</i> device uploaded it off the mesh
location, hasLocation	GeoPoint, bool	hasLocation=false suppresses the map pin instead of drawing (0,0)
confidence	number	Cosine score, clamped to [0, 1]
status	string	pending dispatched accepted dismissed – only the kiosk may change it
timestamp	Timestamp	Must equal request.time; no back-dating

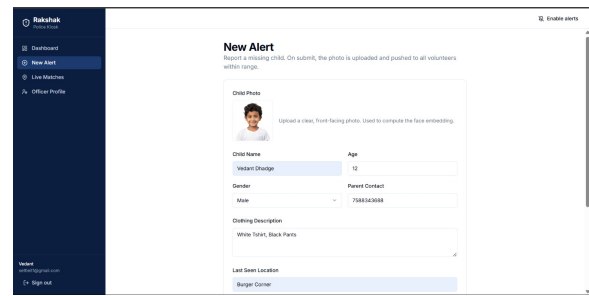
officers/{uid} (kiosk directory) holds uid, email, photoURL, role, displayName, phone, station, badgeNumber, fcmToken?, createdAt, lastLoginAt, updatedAt and is written once by claimOfficerRole (Listing 7.1); the browser may only patch displayName/phone/station/badgeNumber/fcmToken. volunteers/{uid} (mobile directory) holds phone, role, name, email, photoUrl, fcmToken, registeredAt, lastLocation, locationUpdatedAt and is written by its own device only. One composite index, alerts(status ASC, timestamp DESC), covers both the kiosk dashboard and the volunteer home screen’s active-alert query.

7.2 Police Kiosk Portal

Built with Next.js 14 (App Router), the kiosk exposes five guarded routes – /login, / (dashboard), /alerts/new, /alerts/history, /matches, and /profile – behind a shell that blocks all protected content until the signed-in account’s police custom claim has been confirmed, so no officer-only data ever flashes on screen for an unauthorised viewer.



(a) Live dashboard: stat tiles, active alerts, match feed



(b) New Alert form – photo, details, geolocation

Figure 7.1: Police kiosk portal

Authentication and authorisation

Both the kiosk and the mobile app share a single Firebase Authentication pool, but authentication (proving who you are) is deliberately kept separate from authorisation (what you may do). Signing in with Google proves identity only; the ability to create an alert is gated on a police custom claim, granted by the `claimOfficerRole` callable Cloud Function.

Listing 7.1: Officer self-registration – excerpt of `functions/src/officers.ts`

```
1 export const claimOfficerRole = onCall(async (request) => {
2   const uid = requireAuth(request);
3   const volunteerDoc = await db.doc('volunteers/${uid}').get();
4
5   if (volunteerDoc.exists) {
6     // One account, one role: refuse and revoke any stale claim.
7     await auth.setCustomUserClaims(uid, { police: false });
8     await auth.revokeRefreshTokens(uid);
9     throw new HttpsError("permission-denied",
10      "This account is already registered as a volunteer.");
11   }
12
13   await auth.setCustomUserClaims(uid, { police: true });
14   await db.doc('officers/${uid}').set({
15     uid, role: "police", createdAt: FieldValue.serverTimestamp(),
16     lastLoginAt: FieldValue.serverTimestamp(),
17   }, { merge: true });
18 });
```

7.2.1 Alert Creation and the Kiosk Data API

All Firestore access on the web side is funnelled through two thin modules – `lib/firestore.ts` and `lib/officers.ts` – rather than scattering `onSnapshot/addDoc` calls through components. This keeps the security-relevant write shape (which fields a client is allowed to send) in one place, one call site per operation:

Table 7.3: Kiosk data API surface (`lib/firestore.ts`, `lib/officers.ts`)

Function	Does
<code>uploadChildPhoto</code>	Sanitises the filename, uploads to <code>alert_images/</code> , returns the download URL
<code>createAlert</code>	Writes the alerts document; captures <code>navigator.geolocation</code> if the browser permits it
<code>resolveAlert</code>	Flips status to resolved, triggering <code>onAlertResolved</code> (§7.5.1)
<code>subscribeActiveAlerts</code> , <code>subscribeAllAlerts</code>	Live <code>onSnapshot</code> listeners behind the dashboard and history page
<code>subscribeMatches</code> , <code>fetchMatchCounts</code>	Live 100-doc match feed, plus five parallel <code>getCountFromServer</code> aggregates for exact dashboard tile counts
<code>acceptMatch</code> , <code>dismissMatch</code> , <code>dispatchMatch</code>	Kiosk-only status transitions on a matches document
<code>subscribeOfficer</code> , <code>updateOfficerProfile</code> , <code>saveOfficerFcmToken</code>	Officer directory read/write, restricted to the whitelisted profile fields

The alert-creation form requires a photo, name, age, gender, clothing description, contact number, and last-seen place/date/time; `identifyingMarks` is optional. The upload filename is sanitised with `safeFileName` before it reaches Storage, because the `alert_images/{imageId}` rule matches a single path segment – an unescaped slash in a filename would silently step outside the intended rule.

7.2.2 Match Review and Dispatch

The `/matches` route renders each pending sighting beside the original alert photo, so the officer never has to context-switch to judge a match. A reviewed match locks: once `accept`, `dismiss`, or `dispatch` has been called, the dialog cannot be reopened for that document.

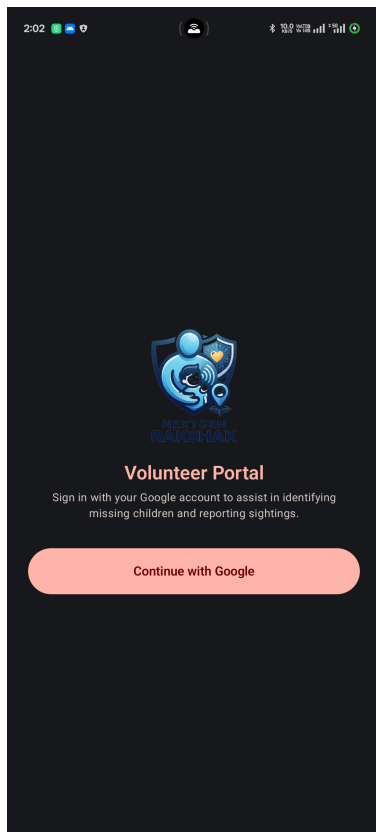
Listing 7.2: Match status transition – excerpt of lib/firestore.ts

```
1 export async function dispatchMatch(matchId: string) {
2   await updateDoc(doc(db, "matches", matchId), {
3     status: "dispatched",
4   });
5   // A Google Maps deep link is built client-side from the match
   's own
6   // GeoPoint -- no extra write, no extra round-trip.
7 }
8
9 export async function dismissMatch(matchId: string) {
10  await updateDoc(doc(db, "matches", matchId), { status: "
    dismissed" });
11 }
```

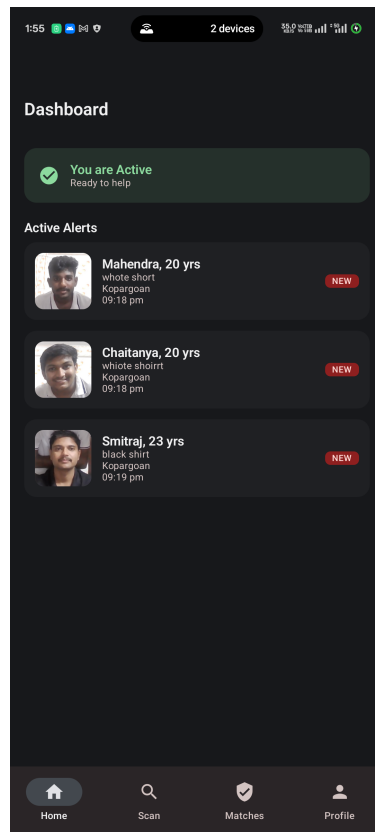
The review dialog additionally: flags a sighting relayed off the mesh (relayedBy set) so the officer knows the reporting volunteer was offline; suppresses the map link entirely when hasLocation is false, rather than plotting a misleading pin at (0,0); and shows the volunteer's self-reported role (police/NCC/NGO/community) so the officer can weigh a sighting's credibility at a glance.

7.3 Volunteer Android Application

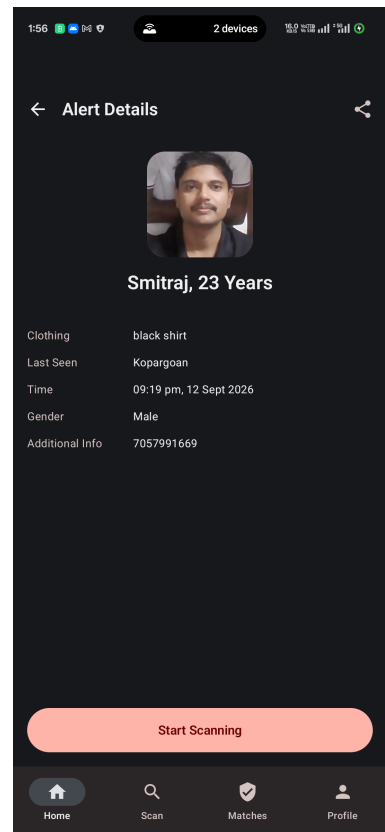
The application follows a clean, SOLID-aligned package layout (data / domain / ml / networking / presentation) wired together with a hand-written ServiceLocator, keeping dependency wiring in one place without the build-time cost of a DI framework on a student project.



(a) Login

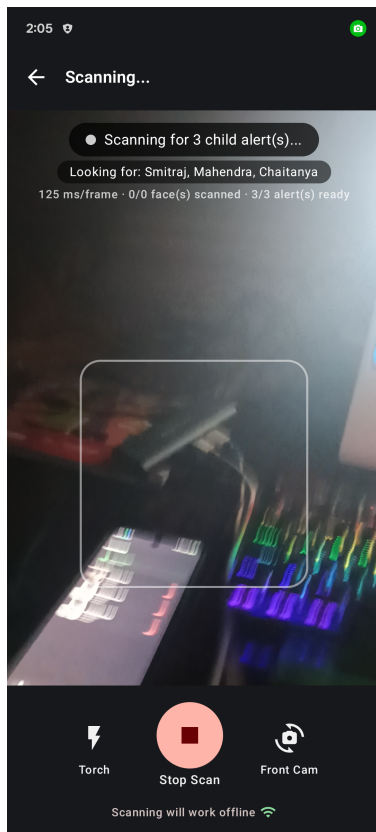


(b) Home – active alerts

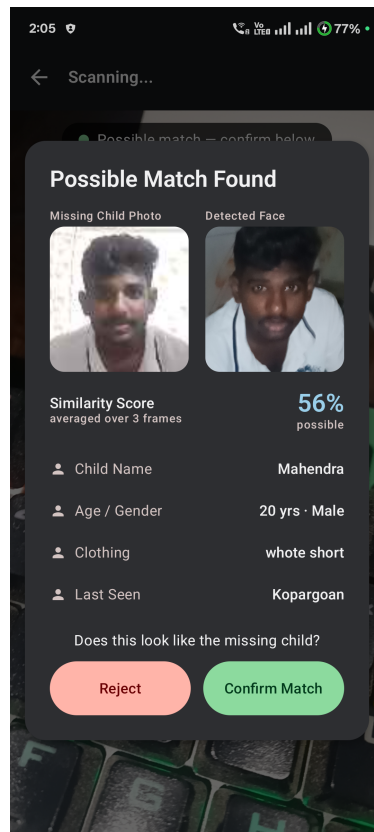


(c) Alert detail

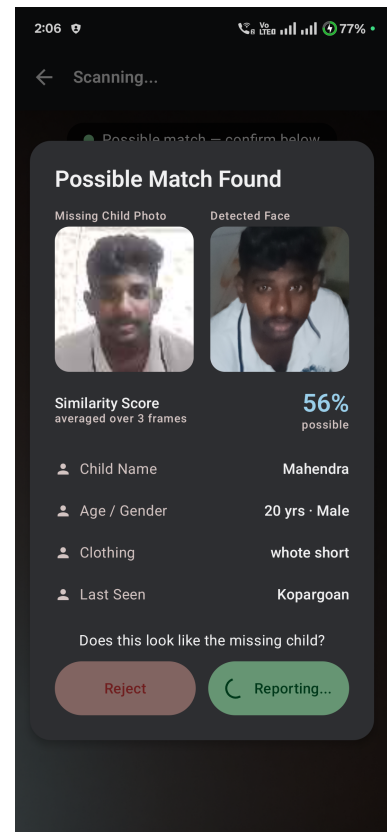
Figure 7.2: Volunteer app – sign-in and alert browsing



(a) Live scan

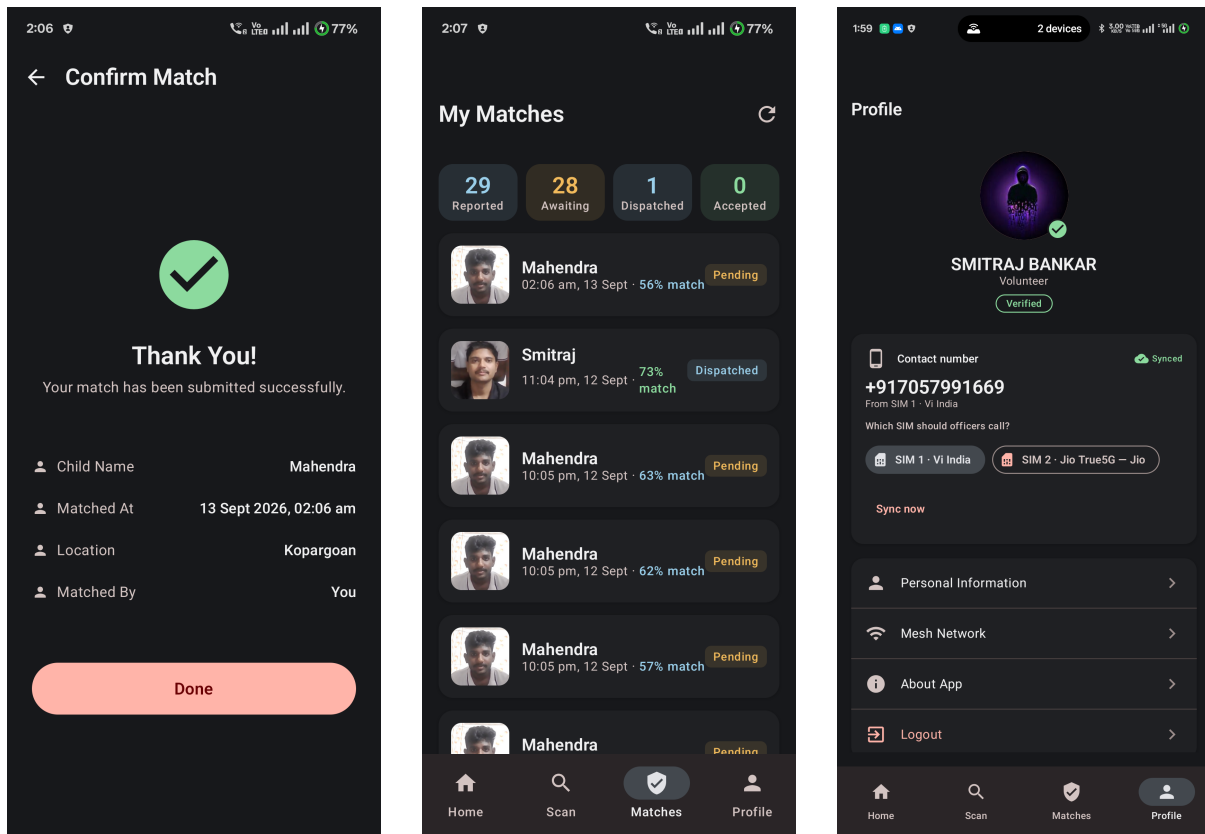


(b) Candidate match dialog



(c) Confirming a match

Figure 7.3: Volunteer app – the core scan-and-confirm flow



(a) Match reported

(b) Volunteer's own match history

(c) Profile

Figure 7.4: Volunteer app – reporting confirmation, history, and profile

7.3.1 Face Recognition Pipeline

The pipeline is deliberately mirrored, field-for-field, across the Android app, the Cloud Function, and the offline evaluation scripts, so that any two implementations produce comparable embeddings:

1. **Detect** – Google ML Kit [12] locates faces and facial landmarks in each camera frame, tagging each detection with a stable tracking ID.
2. **Frontality gate** – faces with yaw beyond 40° or roll beyond 35° are rejected before any further processing.
3. **3-point similarity alignment** – the left eye, right eye, and nose landmarks are warped onto a canonical 112×112 ArcFace [11] template, so pose and scale differences are normalised out before embedding.
4. **Quality gate** (device only) – faces smaller than 48 px, or with mean luma outside 25–240, or a Laplacian-variance sharpness below 12, are rejected as too small, too dark/bright, or too blurred to embed reliably.

5. **Embed** – the aligned $112 \times 112 \times 3$ tile is normalised to $[-1, 1]$ and passed through the MobileFaceNet [9] TFLite [13] interpreter, producing an L2-normalised embedding vector.
6. **Multi-frame fusion** – up to three embeddings per tracked face are averaged, gated by a coherence floor so a recycled tracking ID cannot blend two different people’s embeddings.
7. **Cosine compare** – the fused embedding is compared against every currently active alert’s embedding.

Listing 7.3: Cosine similarity comparator – domain/matching/CosineEmbeddingComparator.kt

```

1 class CosineEmbeddingComparator : EmbeddingComparator {
2     override fun similarity(a: FloatArray, b: FloatArray): Float
3     {
4         require(a.size == b.size) { "Embedding width mismatch" }
5         var dot = 0f
6         for (i in a.indices) dot += a[i] * b[i]
7         // Both vectors are L2-normalised at embedding time,
8         // so the dot product alone equals cosine similarity.
9         return dot.coerceIn(-1f, 1f)
10    }
11 }

```

Listing 7.4: Match decision constants – utils/Constants.kt (excerpt)

```

1 object Constants {
2     const val SIMILARITY_THRESHOLD = 0.55f           // measured, not
3     assumed
4     const val STRONG_MATCH_THRESHOLD = 0.72f         // instant
5     single-frame match
6     const val MATCH_CONFIRM_FRAMES = 2              // mid-band
7     confirmation
8     const val EMBEDDING_FUSION_FRAMES = 3
9     const val TRACK_COHERENCE_MIN = 0.5f
10    const val FACE_CROP_MARGIN = 0.2f
11 }

```

7.3.2 Alert embeddings are recomputed on-device

A key correctness fix discovered during testing: the phone re-embeds the alert’s own photograph with its own model at scan time, rather than trusting the server’s embedding directly. Measurement showed that comparing an embedding produced by one geometry pipeline (a centred crop)

against one produced by another (3-point alignment) scores the *same face* at 0.38–0.49 – below the 0.55 threshold – even though the two photos are of the same person. Mixing geometries does not degrade matching gracefully; it silently disables it. Re-embedding on-device removes this failure class entirely, since both sides of every comparison now come from one implementation.

7.3.3 Face-Match Threshold: Measured, Not Assumed

The synopsis originally assumed a cosine threshold of 0.75, taken from general face-recognition literature rather than from this model on this pipeline. It was re-measured directly, and the measurement changed the number:

Table 7.4: Threshold measurement – 36 labelled real pairs, original unaligned MobileFaceNet

Same-person pairs	Different-person pairs
Cosine score range: 0.7142 – 0.9899	Cosine score range: 0.0864 – 0.3551

The two ranges leave an empty band from 0.3551 to 0.7142 with no observed pairs at all; `SIMILARITY_THRESHOLD = 0.55` was set near the middle of that gap. The synopsis’s original 0.75 sits *inside* the genuine-pair range above – at that value the system would have rejected several true matches as non-matches. This is the concrete reason the team’s engineering policy is to re-measure after any change to the model, the alignment, or the TFLite precision, rather than trust a literature default – a policy this exact number already justified once.

Table 7.5: Geometry parity – same photo, same identity, different alignment pipelines

Pairing	Cosine score
Centred crop vs. centred crop	0.8855
3-point alignment vs. 3-point alignment	0.8467
Centred crop vs. 3-point alignment (<i>same</i> photo, mismatched geometry)	0.38 – 0.49

This is the measurement behind §7.3.2’s re-embedding decision: mixing two correct alignment geometries scores an *identical* face below the 0.55 threshold. A geometry mismatch does not degrade matching gracefully – it silently disables it, which is far more dangerous than an obvious crash because nothing in the UI indicates a fault has occurred.

7.3.4 Concurrency Safety on the Interpreter Thread

The TFLite interpreter is not safe to call from two frames at once. It runs on a single dedicated thread (XNNPACK, 4 threads internally), and every call into it is gated by an `AtomicBoolean`: if a frame arrives while the previous one is still being embedded, it is dropped rather than queued

behind the interpreter. A race in this exact gate – traceable to commit bb0f829 (Table 4.2) – was the “model deadlock” found during the final week of real-device testing (§4.1.3): under sustained load the interpreter could be entered twice, and the second call would block forever waiting on a lock the first call already held. It surfaced only on physical hardware under continuous scanning, never on the emulator or in a unit test, which is why the fix landed in the project’s final week rather than earlier.

7.3.5 Local Persistence and Background Sync

A confirmed match must survive a dropped connection, a killed process, or a phone that is genuinely offline for the rest of the event. Reports are therefore never held only in memory: ReportMatchUseCase always writes to a Room database first, and network delivery happens asynchronously from that durable queue.

Listing 7.5: Pending-match entity and DAO – data/local/PendingMatchEntity.kt (excerpt)

```

1  @Entity(tableName = "pending_matches")
2  data class PendingMatchEntity(
3      @PrimaryKey val id: String,                // client-generated
4          UUID
5      val alertId: String,
6      val confidence: Float,
7      val latitude: Double?,
8      val longitude: Double?,
9      val sightingPhotoPath: String,
10     val status: String,                        // QUEUED | SYNCED |
11         RELAYED
12     val createdAt: Long,
13 )
14
15 @Dao
16 interface PendingMatchDao {
17     @Insert suspend fun insert(entity: PendingMatchEntity)
18     @Query("SELECT * FROM pending_matches WHERE status = 'QUEUED'")
19     suspend fun queued(): List<PendingMatchEntity>
20     @Update suspend fun update(entity: PendingMatchEntity)
21 }

```

MatchSyncWorker, a connectivity-gated WorkManager job, drains this queue whenever the device regains a network path – including a path that only just opened because a mesh peer came online. AppDatabase is versioned (currently v4); each schema change (adding the mesh tables,

then the thumbnail column, then the gateway-routing fields) shipped its own Migration, so an upgrading install never loses a volunteer's already-queued reports.

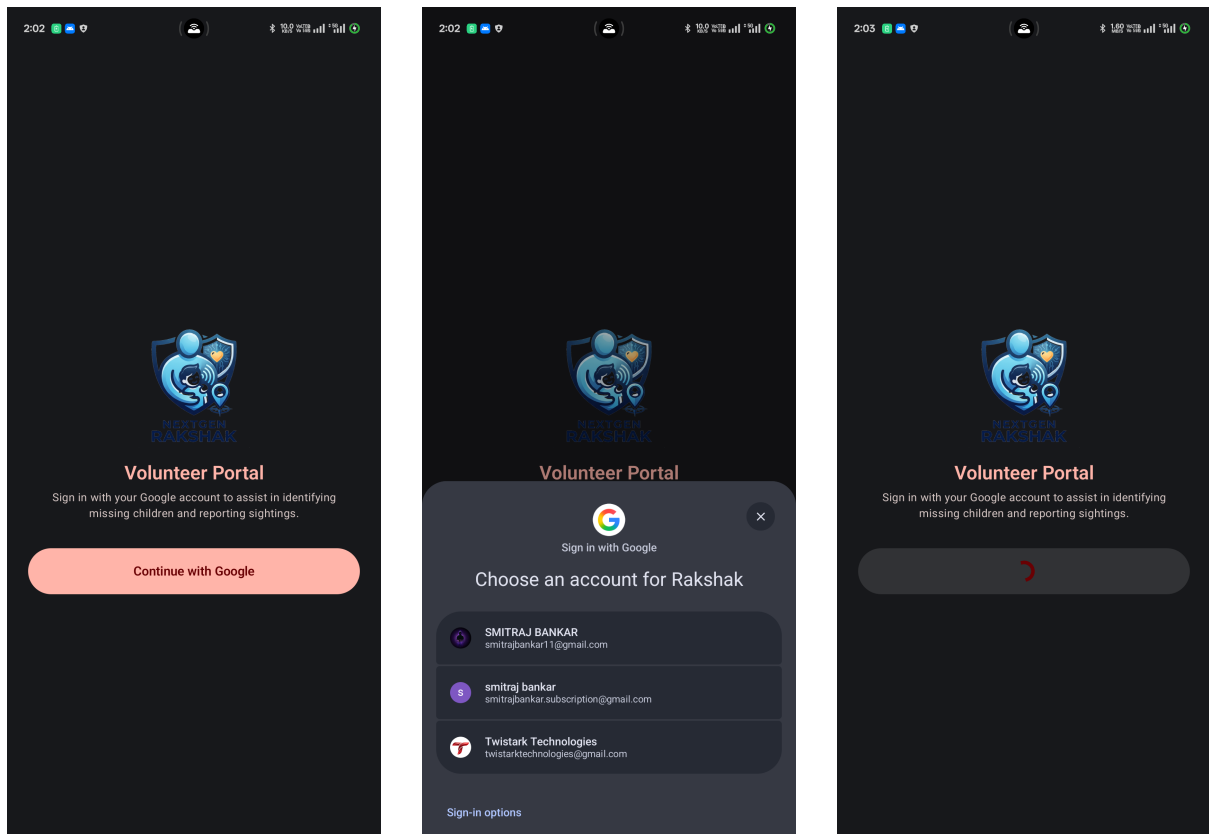
Listing 7.6: Connectivity-gated sync – networking/MatchSyncWorker.kt (excerpt)

```
1 class MatchSyncWorker(ctx: Context, params: WorkerParameters) :  
2     CoroutineWorker(ctx, params) {  
3  
4     override suspend fun doWork(): Result {  
5         val queued = pendingMatchDao.queued()  
6         queued.forEach { pending ->  
7             val uploaded = matchRepository.upload(pending)  
8             if (uploaded) pendingMatchDao.update(pending.copy(  
9                 status = "SYNCED"))  
10        }  
11        return Result.success()  
12    }  
13  
14    companion object {  
15        fun enqueue(ctx: Context) {  
16            val constraints = Constraints.Builder()  
17                .setRequiredNetworkType(NetworkType.CONNECTED)  
18                .build()  
19            val request = OneTimeWorkRequestBuilder<  
20                MatchSyncWorker>()  
21                .setConstraints(constraints)  
22                .build()  
23            WorkManager.getInstance(ctx).enqueue(request)  
24        }  
25    }  
26 }
```

The volunteer's own profile (name, phone, cached avatar URL) is kept in a separate DataStore-backed VolunteerStore, not Room – a single small object with no query needs, refreshed from the SIM (SimPhoneProvider) every time the Profile screen opens so the number an officer would call is always current.

7.4 Complete End-to-End Mobile Application Workflow

To document the volunteer app as it actually behaves rather than in isolated fragments, Figures 7.5–7.11 walk through all nineteen screens of one continuous session on real hardware, in the exact order they occur: signing in, receiving and reading an alert, scanning and confirming a live match, reviewing match history, managing the profile and the offline mesh, and signing out.

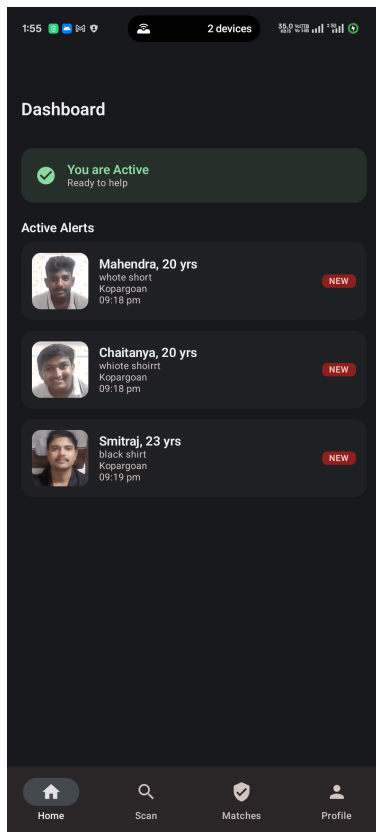


(a) 1. Login screen

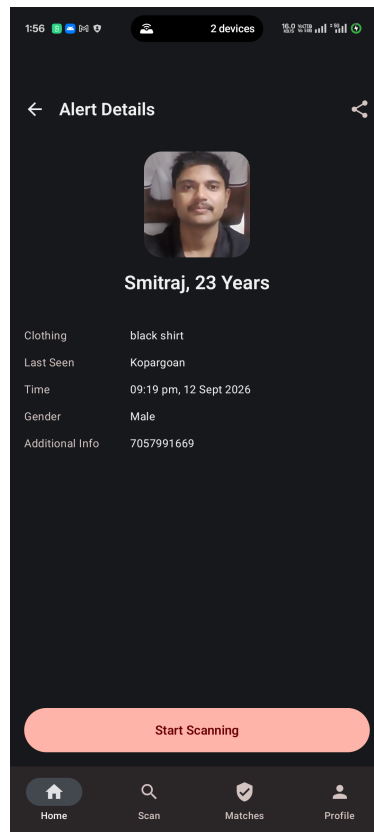
(b) 2. Google account picker

(c) 3. Signing in

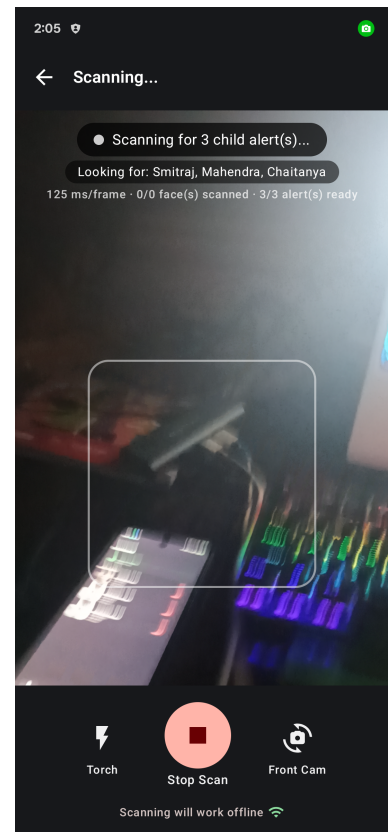
Figure 7.5: Mobile workflow, screens 1–3: Google sign-in



(a) 4. Home dashboard

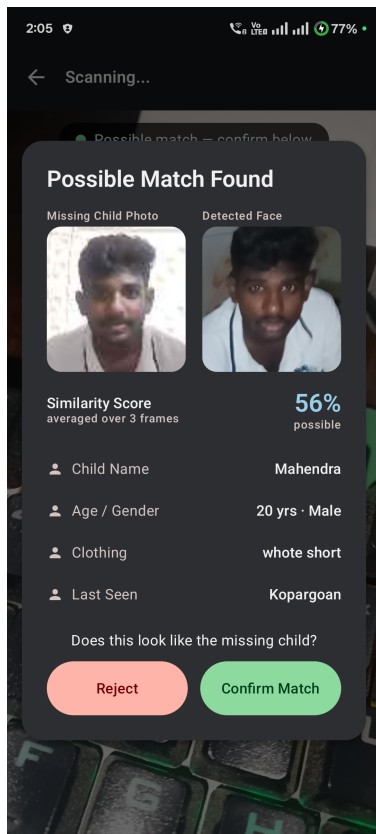


(b) 5. Alert details

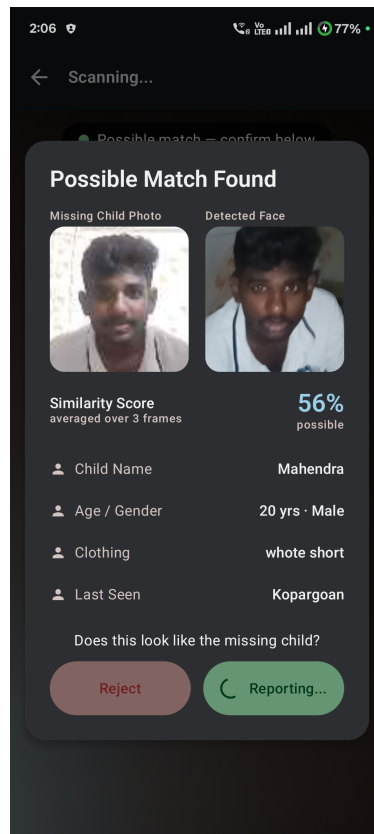


(c) 6. Live camera scan

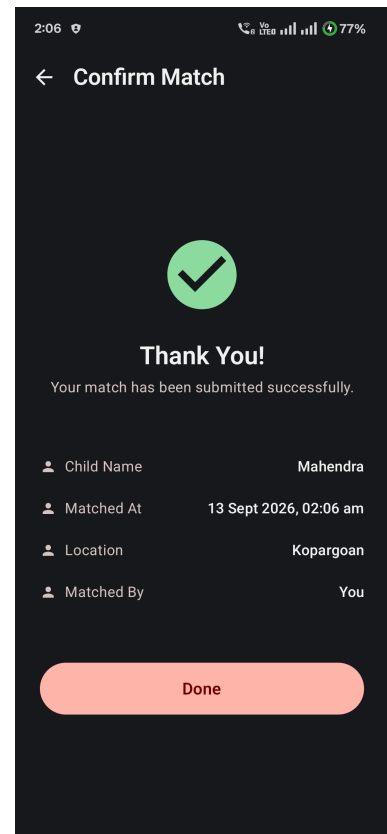
Figure 7.6: Mobile workflow, screens 4–6: home, alert detail, and live scan



(a) 7. Possible-match popup

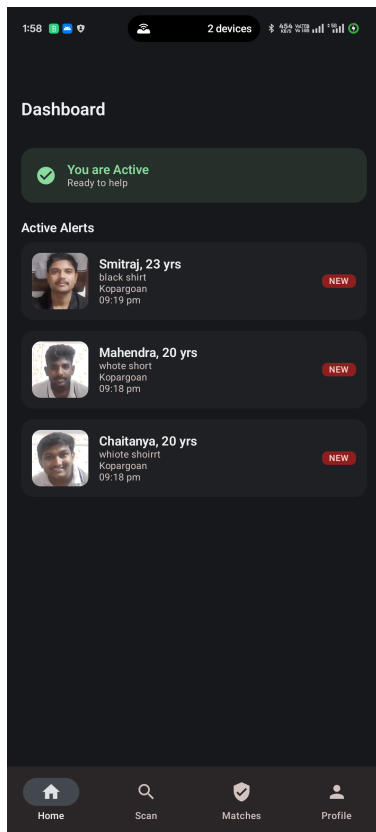


(b) 8. Confirming the match

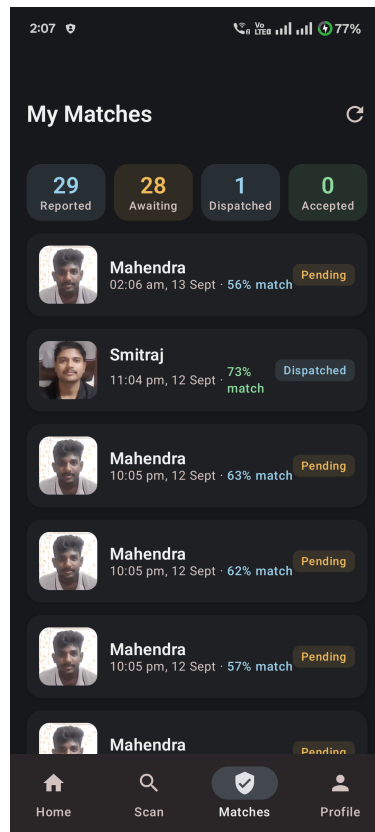


(c) 9. Match submitted

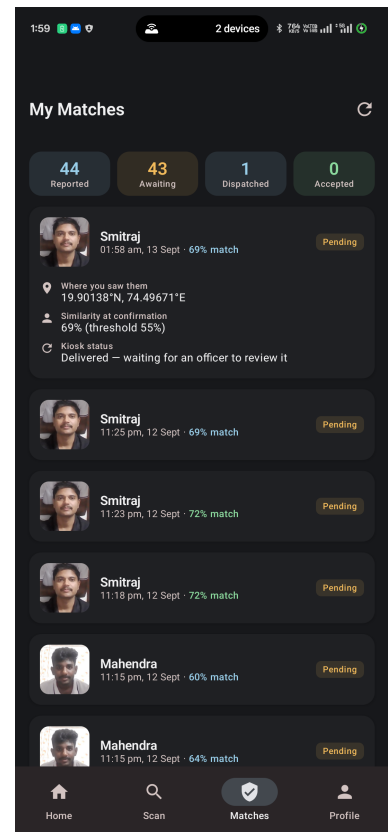
Figure 7.7: Mobile workflow, screens 7–9: candidate match through confirmation



(a) 10. Home after match

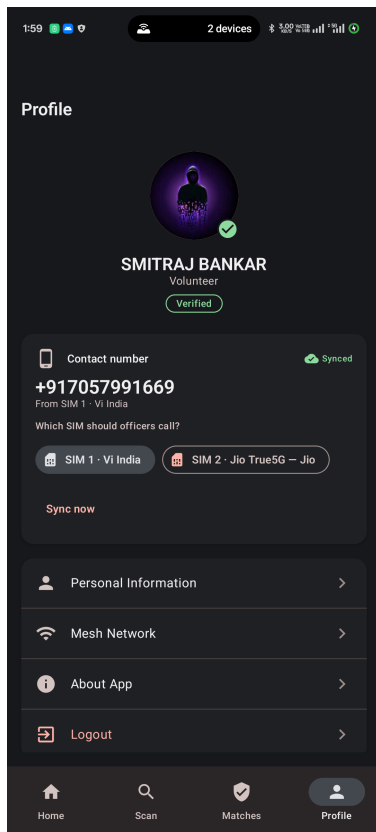


(b) 11. Matches list

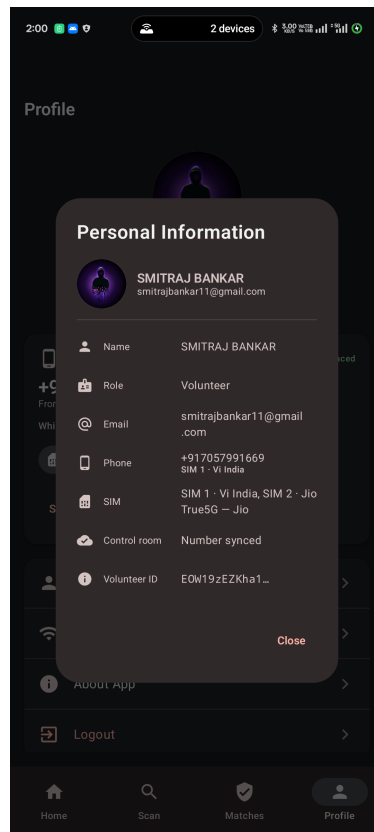


(c) 12. Match – expanded detail

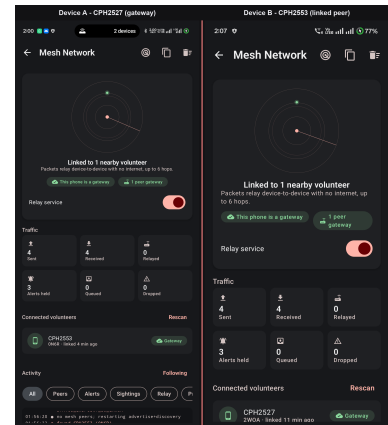
Figure 7.8: Mobile workflow, screens 10–12: post-match home and match history



(a) 13. Profile

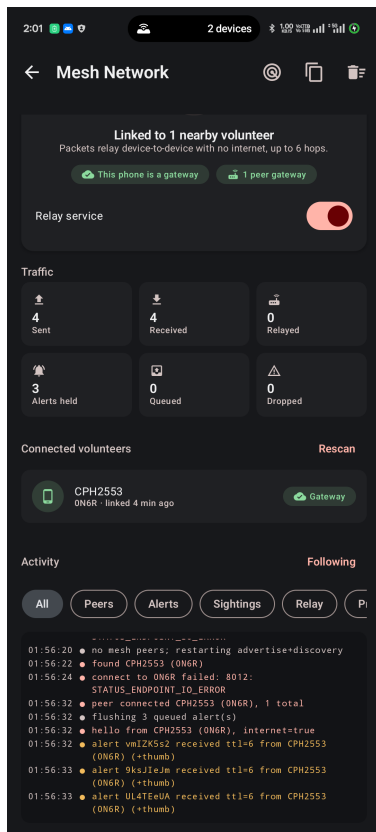


(b) 14. Personal info

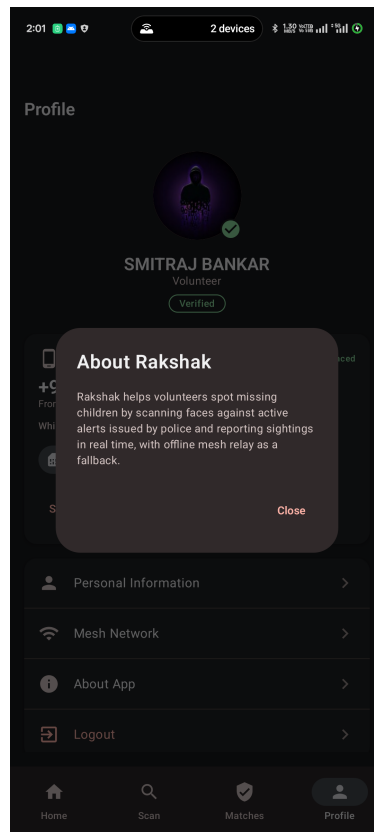


(c) 15. Mesh network view

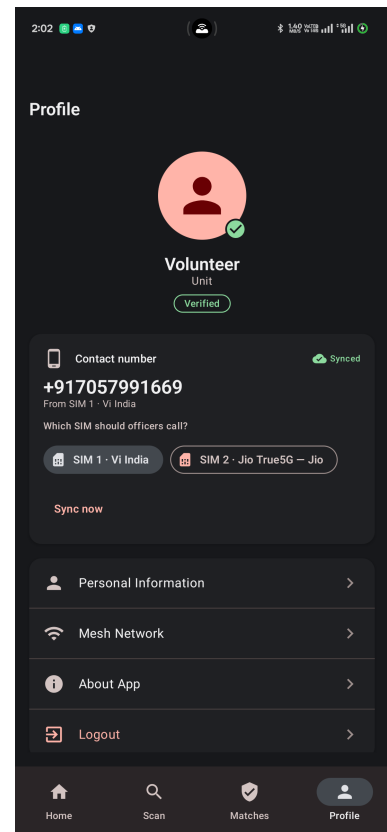
Figure 7.9: Mobile workflow, screens 13–15: profile, personal info, and mesh screen



(a) 16. Mesh activity log

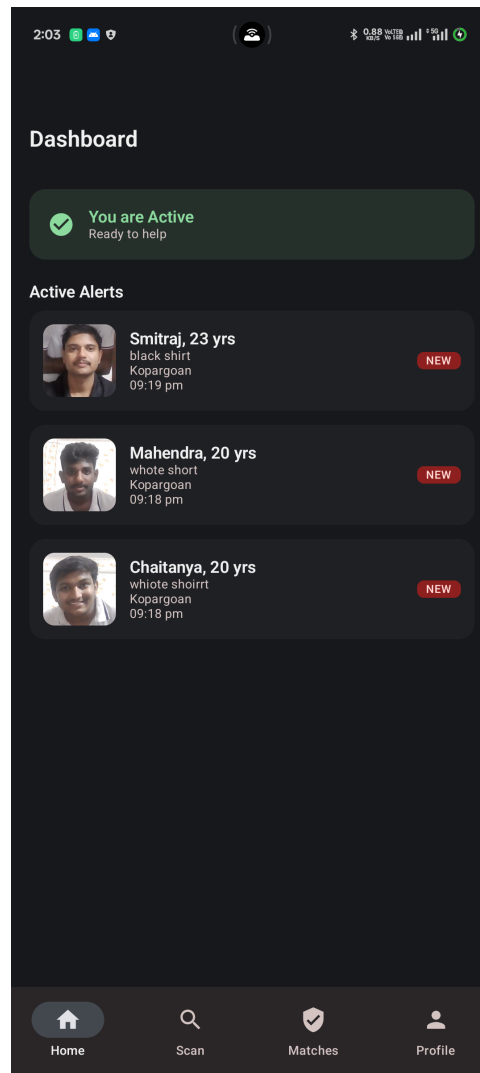


(b) 17. About app



(c) 18. Logout tapped

Figure 7.10: Mobile workflow, screens 16–18: mesh log, about, and sign-out



(a) 19. Back at login after sign-out

Figure 7.11: Mobile workflow, screen 19: session end – state restored to login

7.5 Cloud Functions and Data Coordination

Coordination between the kiosk and every volunteer device runs entirely on Firebase [14] – Firestore, Cloud Storage, Authentication, and Cloud Messaging – with six Cloud Functions doing the server-side work neither client can do on its own.

Table 7.6: Cloud Function exports and their trigger

Function	Trigger	Responsibility
onAlertCreated	Firestore create on alerts	Fallback embedding computation, geofenced FCM broadcast
onMatchCreated	Firestore create on matches	Push the confirmed sighting to the filing officer
onAlertResolved	Firestore update, active→resolved	Purge embedding, alert photo, and every copied match photo URL
expireAlerts	Scheduled, every 30 min	Auto-resolve alerts older than the configured TTL (8 h)
compute-EmbeddingCallable	Callable (officers only)	Re-index one alert's embedding on demand
claimOfficerRole	Callable	Grant the police claim and upsert the officer directory record

Geofenced broadcast

Listing 7.7: Haversine geofence filter – excerpt of functions/src/notify.ts

```

1  const GEOFENCE_RADIUS_KM = 2;
2  const STALE_LOCATION_MS = 6 * 60 * 60 * 1000;
3
4  function withinGeofence(alertLoc: GeoPoint, volunteerLoc:
      GeoPoint): boolean {
5      const R = 6371; // Earth radius, km
6      const dLat = toRad(volunteerLoc.latitude - alertLoc.latitude);
7      const dLon = toRad(volunteerLoc.longitude - alertLoc.longitude
          );
8      const a = Math.sin(dLat / 2) ** 2 +
9          Math.cos(toRad(alertLoc.latitude)) * Math.cos(toRad(
              volunteerLoc.latitude)) *
10         Math.sin(dLon / 2) ** 2;
11     const distanceKm = 2 * R * Math.asin(Math.sqrt(a));
12     return distanceKm <= GEOFENCE_RADIUS_KM;
13 }
14 // Fail-open: a volunteer is skipped only if their fix is valid
    AND fresh
15 // AND out of range -- a stale or missing fix never silently
    excludes them.

```

7.5.1 Data Lifecycle – Privacy Purge on Resolve

This is where the project’s privacy claim becomes enforceable code rather than a policy statement. The moment an officer resolves a case (or the scheduled 8-hour sweep does it for them), `onAlertResolved` deletes the Storage object outright and clears every trace of the biometric vector:

Listing 7.8: Purge on resolve – excerpt of `functions/src/index.ts`

```
1 export const onAlertResolved = onDocumentUpdated("alerts/{
    alertId}", async (event) => {
2   const before = event.data?.before.data();
3   const after = event.data?.after.data();
4   if (before?.status !== "active" || after?.status !== "resolved"
        ") return;
5
6   await storage.bucket().file(extractPath(after.imageUrl)).
        delete().catch(() => {});
7   await event.data!.after.ref.update({ embedding: [], imageUrl:
        "" });
8
9   // Every match copied this alert's photo URL for its own
       listener --
10  // blank those too, or the kiosk shows a permanently broken
       image.
11  const matches = await db.collection("matches")
12    .where("alertId", "==", event.params.alertId).get();
13  const batches = chunk(matches.docs, 500);           // Firestore
       batch limit
14  for (const b of batches) {
15    const batch = db.batch();
16    b.forEach((m) => batch.update(m.ref, { imageUrl: "" }));
17    await batch.commit();
18  }
19  });
```

`expireAlerts` (a 30-minute scheduled function) reuses the same chunked-batch pattern to flip every alert past its TTL to resolved – which in turn fires `onAlertResolved` for each one, so a backlog of stale alerts cannot throw partway through and leave some of them holding live biometric data.

7.6 Offline Mesh Networking

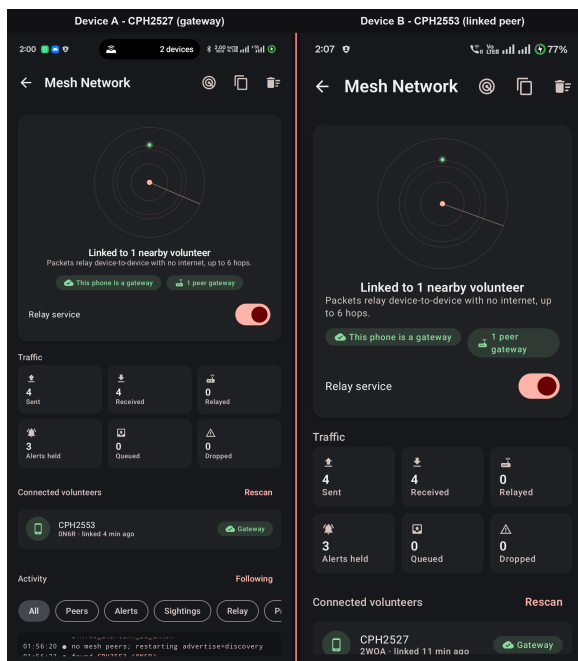
Google’s Nearby Connections API [10] links pairs of nearby devices over Bluetooth Low Energy (discovery) and Wi-Fi Direct (transfer) – it is not itself a multi-hop mesh protocol. NextGen Rakshak implements a dedicated application-level store-and-forward layer on top of it.

Table 7.7: Mesh packet wire format

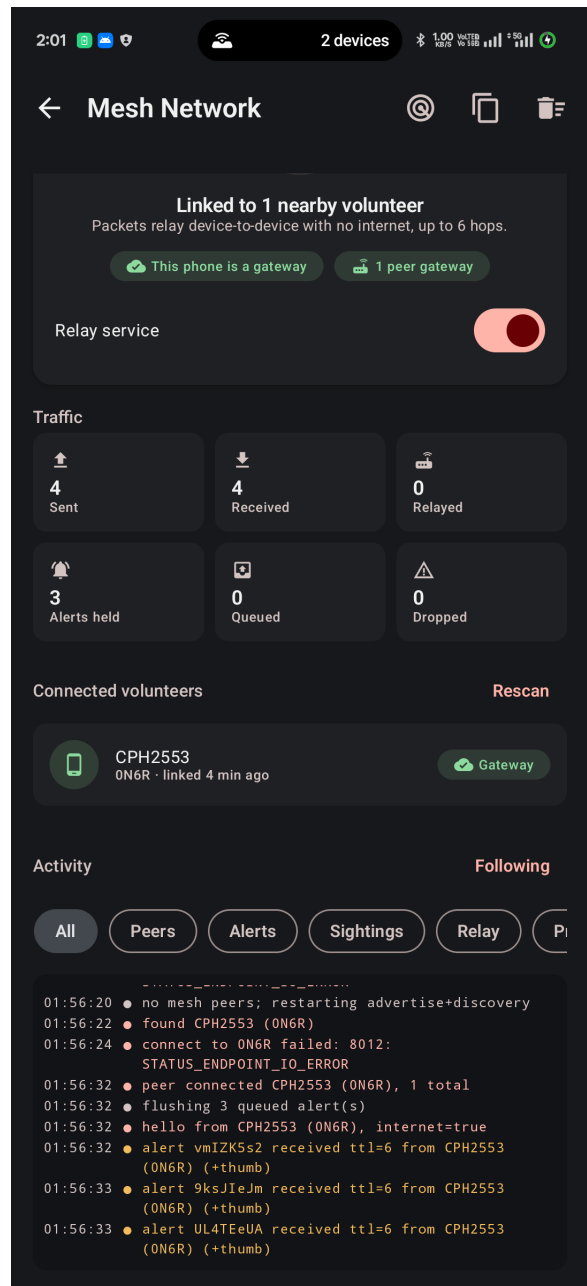
Field	Purpose
TTL (1 byte)	Hop-count, initialised to 6, decremented per relay, packet dropped at 1
Type tag (1 byte)	alert match resolve hello ack
Message ID (UUID)	Uniquely identifies the packet so a device relays it at most once
Payload	Type-specific fields: alert text fields, embedding vector, ≤ 8 KB face thumbnail (alert); GPS + confidence (match)
HMAC-SHA256 trailer (32 bytes)	Authenticates everything except the mutable TTL byte; a packet that fails verification is dropped before parsing

Listing 7.9: Relay guard – simplified from `networking/mesh/MeshNetworkManager.kt`

```
1 fun onPacketReceived(raw: ByteArray, fromEndpoint: String) {
2     val packet = MeshPayloadCodec.decode(raw) ?: return
3     // bad HMAC -> drop
4     if (seenCache.contains(packet.messageId)) return
5     // already relayed
6     seenCache.remember(packet.messageId, ttlMillis =
7         ALERT_LIFETIME_MS)
8     handleLocally(packet)
9     // update UI/state
10
11     if (packet.ttl > 1) {
12         val relay = packet.copy(ttl = packet.ttl - 1)
13         connectedEndpoints.filter { it != fromEndpoint }
14             .forEach { sendPayload(it, MeshPayloadCodec.encode(
15                 relay)) }
16     }
17 }
```



(a) Mesh network screen – peer status



(b) Mesh activity log – packet-level trace

Figure 7.12: Offline mesh – live visualisation on the volunteer app

A three-physical-device field trial confirmed genuine multi-hop relay: an alert was traced hopping two links deep (TTL decrementing $6 \rightarrow 5$) between two phones that were never directly paired, over a star topology formed by Nearby’s own discovery timing. The trial also surfaced and fixed two reconnect-stability defects invisible to unit tests – a missing run-time NEARBY_WIFI_DEVICES permission request on Android 13+, and a discovery-only “endpoint lost” callback that was incorrectly wired to the same logic as a genuine disconnect, zeroing the peer count for a link that was in fact still alive.

7.7 Security Implementation

Firestore security rules enforce every authorisation boundary server-side – the client-side route guard is a UX convenience, not the security boundary.

Listing 7.10: Firestore rule excerpt – alerts and matches (firestore.rules)

```
1 match /alerts/{alertId} {
2   allow read: if request.auth != null;
3   allow create: if isPolice() && request.resource.data.createdBy
4     .uid == request.auth.uid;
5   allow update: if isPolice();
6   allow delete: if false;
7 }
8 match /matches/{matchId} {
9   allow read: if request.auth != null;
10  allow create: if request.auth != null && validMatch() &&
11    attributed();
12  allow update: if isPolice() &&
13    request.resource.data.diff(resource.data).affectedKeys().
14      hasOnly(['status']) &&
15    request.resource.data.status in ['pending', 'dispatched', '
16      accepted', 'dismissed'];
17  allow delete: if false;
18 }
```

7.8 Deployment

The police kiosk portal is deployed and publicly reachable at <https://nextgen-rakshak.vercel.app/> – this is not a local development build; it is the same production build the testing in §7.9.4 and Table 7.9 was carried out against. Each of the project’s three components is deployed separately, matching the three-layer architecture in Figure 4.9.

7.8.1 Police Kiosk Portal – Vercel

The kiosk is a standard Next.js 14 App Router project, which Vercel builds with zero custom configuration.

1. The GitHub repository (§4.1.2) is connected to a Vercel project, with the project root set to:
nextgen-rakshak-webportal/.

2. The Firebase web SDK configuration is supplied as environment variables in the Vercel project settings – never committed to the repository – covering the API key, auth domain, project ID, storage bucket, messaging sender ID, app ID, and the Cloud Messaging VAPID key the notification bell needs for push.
3. Vercel runs `next build` against these variables and deploys the result to its edge network; every push to main triggers a fresh production deployment automatically, with no manual step.
4. The Firebase project's Authentication settings are configured to accept the deployed domain (`nextgen-rakshak.vercel.app`) as an authorised OAuth redirect origin for Google sign-in.

7.8.2 Cloud Functions, Firestore Rules, and Storage Rules – Firebase

The backend is deployed independently of the kiosk, directly from the `functions/` workspace, via the Firebase CLI:

Listing 7.11: Backend deployment command

```
1 cd functions && npm install
2 firebase deploy --only functions,firestore:rules,firestore:
  indexes,storage:rules
```

This single command pushes all six Cloud Functions (Table 7.6), the Firestore security rules (§7.7), the composite index, and the Storage rules together, so the rules and the functions that assume them are always deployed as one atomic unit rather than drifting apart.

7.8.3 Volunteer Android App – Signed Release APK

The volunteer app is not distributed through the Play Store; it is sideloaded as a signed release APK, which is the standard distribution model for a final-year field trial.

1. `google-services.json` for the same Firebase project is placed in the mobile app's module directory:
`nextgen-rakshak-mobile/app/`.
2. The release signing key's SHA-1 is registered against the Firebase Android app, and `MESH_HMAC_KEY` is set in `local.properties` – a release build refuses to compile with the well-known development key (§7.6), so this step cannot be silently skipped.
3. `./gradlew :app:assembleRelease` produces the signed APK, which was installed directly on the field-trial devices used in §7.9.4.

All three deployments share one Firebase project, so a change to the Firestore schema or security rules takes effect for the live kiosk and every installed copy of the app simultaneously.

7.9 Testing, Field Validation, and Results

7.9.1 Testing Strategy

Four layers of testing were used, matching the four things that can independently break in a system this shape: the matching logic itself, the mesh’s packet handling, the full stack wired together, and the stack running on real, physically separate hardware rather than an emulator.

- **Unit testing** – the matching and mesh-networking logic, which is pure computation with no Android framework dependency, is covered by the JVM test suite in §7.9.2.
- **Integration testing** – each Cloud Function was exercised against a live Firestore project (alert create → embedding → geofenced push → match create → officer push), and the kiosk’s data-API layer (§7.2.1) against the deployed rules.
- **System testing** – the full alert-to-dispatch path was walked manually on real Android hardware for every functional requirement in Table 2.1 at least once (Table 7.9).
- **Field testing** – the offline mesh was run across physically separate phones outside the emulator, since Nearby Connections’ BLE/Wi-Fi Direct behaviour cannot be simulated (§7.9.4).

7.9.2 Unit Test Suite

The volunteer app’s core matching and mesh logic is covered by unit tests, run through `./gradlew :app:testDebugUnitTest`:

Table 7.8: Unit test suite – volunteer Android app

Matching / geometry	Mesh networking
CosineEmbeddingComparatorTest	MeshCryptoTest
AlertIndexTest	MeshPayloadCodecTest
EmbeddingAggregatorTest	MeshRouterTest
TrackRegistryTest	MeshSeenCacheTest
FaceGeometryTest	MeshAlertSourceTest
ElapsedTimeTest	

7.9.3 System Test Cases

Every row below was executed manually on real Android hardware (not the emulator) against the deployed Firebase project, not a mock backend. Results are reported as observed – a defect *found* during this pass is reported as found, with the fix that followed it, rather than silently retested until it disappears from the table.

Table 7.9: System-level test cases, executed on real hardware

ID	Scenario	Result
TC-01	Officer sign-in is gated on the police custom claim, not merely being signed in (FR)	Pass – a non-claimed Google account is refused; <code>claimOfficerRole</code> grants access on first legitimate sign-in
TC-02	Officer files an alert with photo + structured details (FR-01)	Pass – alerts document created, photo stored, geolocation captured
TC-03	Fallback embedding computed server-side on alert creation (FR-02)	Pass – <code>onAlertCreated</code> writes the embedding within the Cloud Function’s cold/warm response window
TC-04	Geofenced push reaches volunteers within 2 km of the kiosk (FR-03)	Pass – FCM notification observed on the volunteer device end-to-end
TC-05	On-device face detection with the frontality gate rejects extreme head-pose (FR-04)	Pass – verified on real device under normal indoor lighting
TC-06	On-device embedding + cosine comparison against the active alert (FR-05, FR-06)	Pass – a genuine live match was observed at $\sim 77\%$ cosine confidence on real hardware
TC-07	Side-by-side confirmation dialog, including the offline-thumbnail fallback (FR-07)	Pass – dialog renders the parent’s photo beside the live camera crop in both the online and mesh-only paths
TC-08	Volunteer explicitly confirms or dismisses a candidate (FR-08)	Pass – both branches verified; a dismissal returns to scanning without reporting
TC-09	Confirmed sighting reaches the kiosk with a GPS fix (FR-09)	Pass – match appears on the officer’s live feed with location
TC-10	Alert auto-expiry after 8 h (FR-12)	Partial – scheduled sweep deployed and observed to fire; not stopwatched end-to-end against a live 8-hour case
TC-11	2-device offline mesh relay, no internet on either device (FR-10)	Pass – alert and match propagate phone-to-phone; confirmed by unit test and by hand
TC-12	3-device offline mesh relay, multi-hop, no internet anywhere (FR-10)	Pass – see the field trial, §7.9.4

ID	Scenario	Result
TC-13	Regression: comparing a centred-crop embedding against a 3-point-aligned embedding of the <i>same</i> face (geometry mismatch)	Defect found – scored 0.38–0.49, below the 0.55 threshold, for an identical face. Fixed by re-embedding every alert photo on-device (§7.3.2)
TC-14	Sustained continuous scanning under load (interpreter concurrency)	Defect found – a race in the interpreter gate could deadlock a second concurrent call. Fixed ; see §7.3.4
TC-15	Mesh discovery/advertising on a fresh 3-device session (Android 13+ permissions, spurious disconnects)	2 defects found and fixed – missing runtime <code>NEARBY_WIFI_DEVICES</code> request, and a discovery callback wrongly wired to disconnect logic; see §7.9.4
TC-16	50 concurrent active alerts (NFR-07)	Not yet tested – open, requires a dedicated multi-alert load rig
TC-17	Recognition accuracy specifically on children’s faces (NFR-02)	Not yet tested – open; see Future Scope (§9.2) for the ethical/dataset constraint

Result summary: 12 of 17 system-level test cases passed outright, 3 uncovered real defects that were fixed as a direct result of testing (TC-13, TC-14, TC-15 – arguably testing’s most valuable output), 1 is partially verified (TC-10), and 2 remain explicitly open (TC-16, TC-17), matching the same items already flagged open in Table 2.2 – nothing here is reported as passing that is not also reflected there.

7.9.4 Field Trial – Offline Mesh on Three Physical Devices

The single hardest claim in this project to fake and the single most important one to verify honestly is that the offline mesh actually relays across more than one hop on real hardware, since Nearby Connections’ BLE discovery and Wi-Fi Direct transfer behaviour cannot be meaningfully reproduced on an emulator.

Setup: three physical Android phones, airplane mode with only Bluetooth and Wi-Fi enabled (no cellular, no internet on any device), the release APK signed with the production `MESH_HMAC_KEY`, all three in Nearby-discovery range of at least one other device.

What was tested: an alert filed on one device’s mesh queue, left to propagate device-to-device with no direct pairing forced between all three; a confirmed match relayed back the same

way; and the system left running long enough to observe reconnect behaviour, not just a single successful hop.

Result: a relayed alert was traced hopping two links deep – the TTL byte decrementing $6 \rightarrow 5$ across two devices confirmed never to be directly paired with each other – proving genuine multi-hop relay rather than a single Nearby Connections link. The three phones held a stable mesh in a star topology (which device links to which is decided by Nearby's own discovery timing, not by the app) for the length of the trial.

The trial also did what field trials are supposed to do: it found two real defects invisible to both unit tests and single-device testing.

1. `NEARBY_WIFI_DEVICES` was declared in the Android manifest but never actually requested at runtime. On Android 13+, a device without it pre-granted could be *found* by another device's discovery but could never run its own discovery – silently and permanently, with only a permission-denied log line as evidence. Fixed by adding it to the app's runtime permission request.
2. `onEndpointLost` – a discovery-layer callback meaning "this BLE beacon is no longer visible," not a real disconnect – was wired to the same logic as a genuine `ConnectionLifecycleCallback.onDisconnected`. Nearby routinely throttles advertising once two devices are already paired, which fired this callback for links that were, per the system's own connection log, still exchanging keep-alive traffic successfully in both directions. This was silently zeroing the peer count – and the match-relay path with it – for connections that were never actually down. Fixed by decoupling the two callbacks and adding a watchdog that re-arms discovery only when a device genuinely has zero peers.

Both fixes shipped before the mesh was considered field-verified rather than merely unit-tested. The remaining open item is measuring hop-relay *timing* under load with more devices (NFR-05), not whether the mesh holds together at all – that question is answered.

8 Applications of the Project

While designed around the missing-child, golden-hour use case, the underlying architecture – geofenced alert distribution, on-device face matching, and an offline mesh fallback – generalises to several adjacent public-safety scenarios:

- **Religious and cultural mass gatherings** – Kumbh Mela, Dussehra processions, Ganesh Chaturthi visarjan, and urban temple fairs, the primary target scenario, where crowd density and network congestion are both at their peak simultaneously.
- **School and college excursions** – a teacher-in-charge can file an alert for a separated student at a crowded destination (a zoo, an exhibition, a pilgrimage trip) using the same kiosk-style flow with chaperones as volunteers.
- **Large-scale public events** – concerts, sporting events, and political rallies, where organised security staff already play the pre-registered-volunteer role the system assumes.
- **Disaster-response and relief camps** – family separation during evacuation is a documented secondary harm in disaster response; the offline mesh is directly applicable where infrastructure damage has taken cellular networks down entirely.
- **Elderly and dementia-patient recovery** – the same photo-embedding-and-match flow applies unchanged to a vulnerable adult who has wandered from a caregiver at a large event.
- **Exit-gate perimeter monitoring** – the optional Raspberry Pi node extends the same on-device matching pipeline to a fixed, unattended camera at a controlled gate, complementing mobile volunteer coverage.

9 Conclusion and Future Scope

9.1 Conclusion

NextGen Rakshak set out to answer a narrow but consequential question: can a system built entirely on smartphones already carried by ordinary volunteers recover a missing child faster than the centralised, internet-dependent, reactive tools currently deployed at Indian mass gatherings, without uploading a single bystander’s face to do it? The project’s central design decision – moving all face recognition on-device and treating the cloud purely as a coordination layer – was carried through consistently across all three components: a Next.js police kiosk for filing and reviewing alerts, a Kotlin/Compose volunteer app that detects, aligns, embeds, and matches faces entirely locally, and Firebase Cloud Functions that route alerts and confirmed sightings without ever seeing a bystander’s image. The kiosk is not merely a local demo build: it is deployed and publicly reachable at <https://nextgen-rakshak.vercel.app/> (§7.8), the same production deployment the field testing in this report was carried out against.

Every functional requirement traced to the six project objectives (Table 2.1) was built, and the great majority were verified on real Android hardware, including the two hardest claims in the literature gap analysis: fully on-device face matching (verified with a live, measured match at $\sim 77\%$ confidence) and genuine multi-hop offline relay across three physically separate devices (verified with a traced, two-hop packet). The face-matching threshold itself was derived from measurement rather than assumed from a paper, and that measurement discipline surfaced and fixed two subtle correctness defects – a geometry-mismatch bug that silently disabled matching, and two mesh-reconnect bugs invisible to unit tests – that would otherwise have undermined the system’s central claims in exactly the conditions a real deployment would expose them.

Equally important to an honest final-year report is what remains open. The non-functional targets in Table 2.2 that require sustained, large-scale field conditions – 50 concurrent alerts, a formal children’s-face accuracy study, and mesh delivery timing beyond three devices – are stated here as unmeasured rather than claimed, consistent with the project’s guiding principle throughout: a documented limitation, explained, is worth more than an unverified claim.

9.2 Future Scope

1. **≥ 3 -device mesh field trial at scale** – extend the three-device trial already completed to 8–10 devices to measure real-world hop-relay timing (NFR-05) rather than a single traced path.
2. **Ethically sourced children’s face dataset** – pursue a licensed academic dataset or IRB-

style consented data collection to validate NFR-02 on the system's actual target population, not an adult proxy.

3. **Per-alert asymmetric signing** – extend the current HMAC-based mesh packet authentication (a shared per-deployment key) with a public-key signature from the kiosk, so a relayed packet's *origin* is verifiable, not merely its integrity.
4. **Concurrency and load testing** – formally benchmark the system against 50 simultaneous active alerts (NFR-07) and 200+ faces/hour per volunteer (NFR-06) under controlled conditions.
5. **512-dimension ArcFace upgrade** – the architecture already reads embedding width from the model at runtime; a higher-capacity ArcFace model could be substituted once its threshold is re-measured per the project's own stated policy.
6. **Raspberry Pi fixed-camera node** – complete the optional exit-gate component descope from the current MVP, reusing the same on-device detection-and-matching pipeline unattended.
7. **Multi-lingual kiosk and app UI** – Indian festivals draw volunteers and officers across many first languages; localisation would materially improve field usability.
8. **Integration with national systems** – a read/write bridge to the government's TrackChild portal [2] would let a case opened in NextGen Rakshak also enter the formal, long-term missing-person record once the golden-hour window has passed.

Bibliography

- [1] National Crime Records Bureau, Ministry of Home Affairs, Government of India, “Crime in India” annual report (missing-persons statistics), as reported via ThePrint, 2023–2024.
- [2] Ministry of Women and Child Development, Government of India, “TrackChild Portal,” <https://www.trackthemissingchild.gov.in>, 2012.
- [3] “Khoya-Paya Portal,” Ministry of Women and Child Development, Government of India, 2015.
- [4] Bachpan Bachao Andolan and Capgemini, “ReUnite – AI-powered facial recognition for missing children,” 2018.
- [5] DSP Mutual Fund and Falco Robotics, “Garuda Rakshak – drone-based child-safety system at Kumbh Mela,” 2025.
- [6] Government of Uttar Pradesh, “Digital Khoya-Paya Kendra – AI-enabled missing-persons centres at Maha Kumbh 2025.”
- [7] Thoothukudi District Police, Tamil Nadu, “Project Guardian – real-time photo-sharing app for festival duty officers,” 2025.
- [8] U.S. Department of Justice, “AMBER Alert Program – national child abduction alert system,” 2025.
- [9] S. Chen, Y. Liu, X. Gao, and Z. Han, “MobileFaceNets: Efficient CNNs for Accurate Real-Time Face Verification on Mobile Devices,” in *Chinese Conference on Biometric Recognition (CCBR)*, 2018.
- [10] Google LLC, “Nearby Connections API – Google Play Services documentation,” <https://developers.google.com/nearby/connections/overview>, 2015.
- [11] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [12] Google LLC, “ML Kit Face Detection – developer documentation,” <https://developers.google.com/ml-kit/vision/face-detection>.
- [13] Google LLC, “TensorFlow Lite – on-device machine learning,” <https://www.tensorflow.org/lite>.

- [14] Google LLC, “Firebase Documentation – Firestore, Cloud Functions, Cloud Messaging,” <https://firebase.google.com/docs>.
- [15] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller, “Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments,” University of Massachusetts, Amherst, Technical Report 07-49, 2007.
- [16] B. W. Boehm, *Software Engineering Economics*, Prentice Hall, 1981.
- [17] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed., McGraw-Hill Education, 2019.